# Predictive Analytics DATA 624 Assignment 1

Mehreen Ali Gillani

2026-01-29

2.10 Exercises

1. Explore the following four time series: Bricks from aus_production, Lynx from pelt, Close from gafa_stock, Demand from vic_elec.

Use ? (or help()) to find out about the data in each series. What is the time interval of each series? Use autoplot() to produce a time plot of each series. For the last plot, modify the axis labels and title.

```
# 1. Explore the data and find out about each series
# Bricks from aus_production
?aus_production

# Lynx from pelt
?pelt


# Close from gafa_stock
?gafa_stock

# Demand from vic_elec
?vic_elec
```

```
# 2. Check the time interval of each series
# Create the series objects
bricks_ts <- aus_production %>% select(Bricks)
lynx_ts <- pelt %>% select(Lynx)
close_ts <- gafa_stock %>% select(Close)
demand_ts <- vic_elec %>% select(Demand)
```

```
print("Checking tsibble structure:")
```

```
## [1] "Checking tsibble structure:"
```

```
print(bricks_ts)
```

```
## # A tsibble: 218 x 2 [1Q]
##    Bricks Quarter
##     <dbl>   <qtr>
##  1    189 1956 Q1
##  2    204 1956 Q2
##  3    208 1956 Q3
##  4    197 1956 Q4
##  5    187 1957 Q1
##  6    214 1957 Q2
##  7    227 1957 Q3
##  8    222 1957 Q4
```

```
##  9    199 1958 Q1
## 10    229 1958 Q2
## # i 208 more rows
```

```r
print(lynx_ts)
```

```
## # A tsibble: 91 x 2 [1Y]
##     Lynx  Year
##    <dbl> <dbl>
##  1 30090  1845
##  2 45150  1846
##  3 49150  1847
##  4 39520  1848
##  5 21230  1849
##  6  8420  1850
##  7  5560  1851
##  8  5080  1852
##  9 10170  1853
## 10 19600  1854
## # i 81 more rows
```

```r
print(close_ts)
```

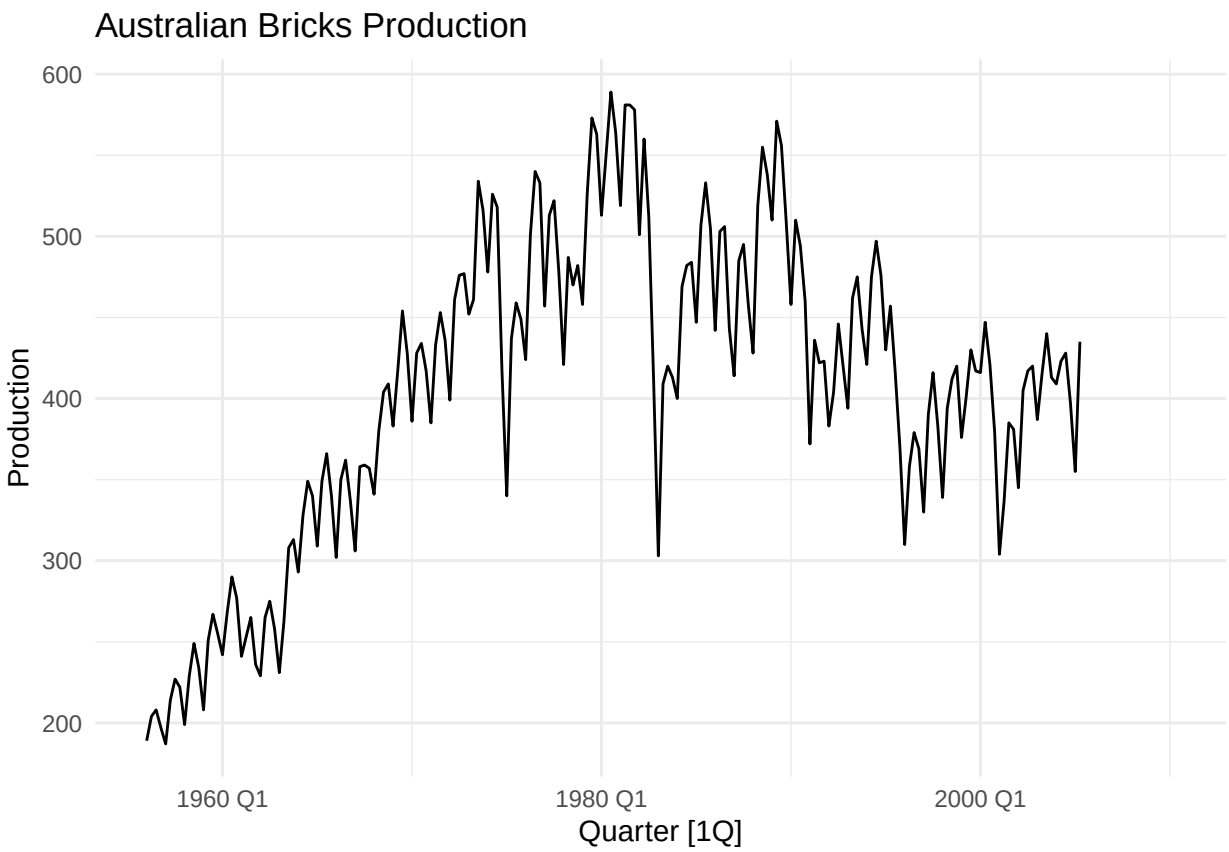
```
## # A tsibble: 5,032 x 3 [!]
## # Key:       Symbol [4]
##    Close Date       Symbol
##    <dbl> <date>     <chr>
##  1  79.0 2014-01-02 AAPL
##  2  77.3 2014-01-03 AAPL
##  3  77.7 2014-01-06 AAPL
##  4  77.1 2014-01-07 AAPL
##  5  77.6 2014-01-08 AAPL
##  6  76.6 2014-01-09 AAPL
##  7  76.1 2014-01-10 AAPL
##  8  76.5 2014-01-13 AAPL
##  9  78.1 2014-01-14 AAPL
## 10  79.6 2014-01-15 AAPL
## # i 5,022 more rows
```

```r
print(demand_ts)
```

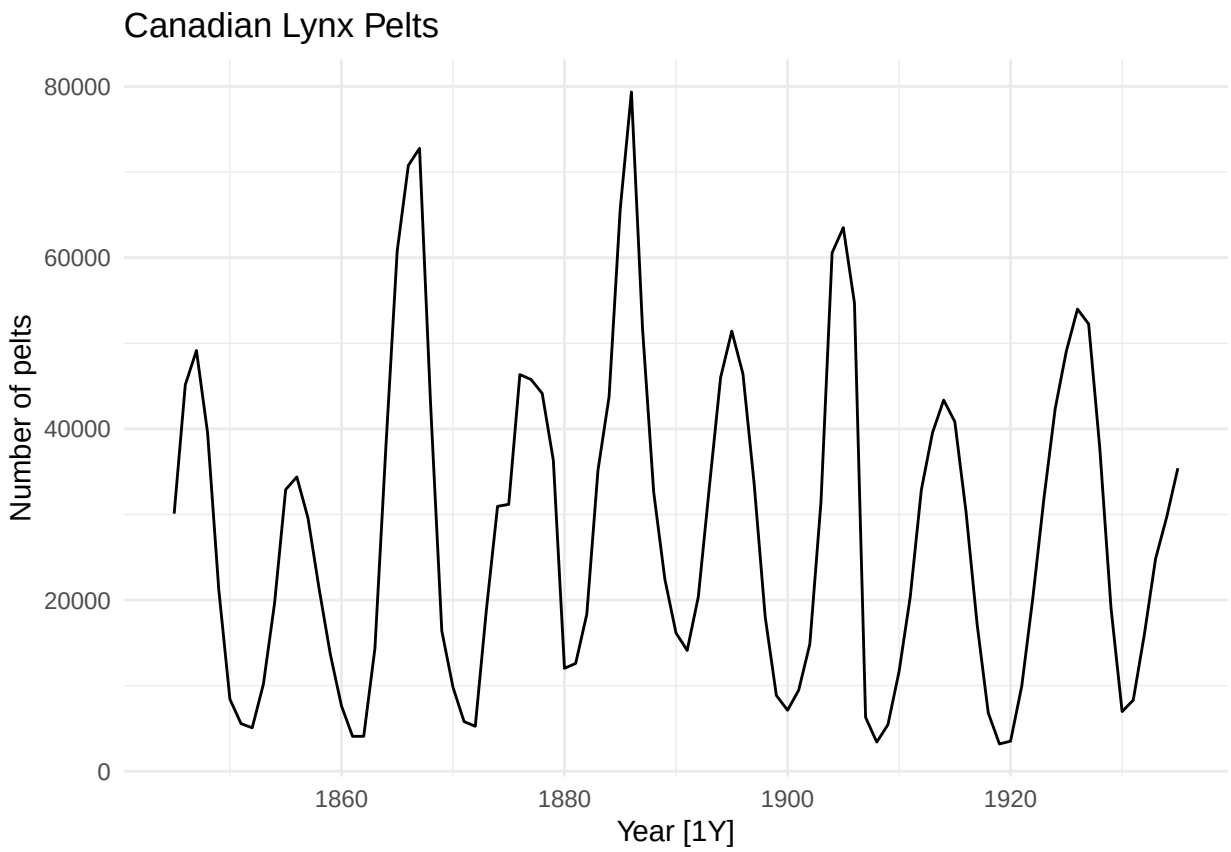
```
## # A tsibble: 52,608 x 2 [30m] <Australia/Melbourne>
##    Demand Time
##     <dbl> <dttm>
##  1  4383. 2012-01-01 00:00:00
##  2  4263. 2012-01-01 00:30:00
##  3  4049. 2012-01-01 01:00:00
##  4  3878. 2012-01-01 01:30:00
##  5  4036. 2012-01-01 02:00:00
##  6  3866. 2012-01-01 02:30:00
##  7  3694. 2012-01-01 03:00:00
##  8  3562. 2012-01-01 03:30:00
##  9  3433. 2012-01-01 04:00:00
## 10  3359. 2012-01-01 04:30:00
## # i 52,598 more rows
```

```r
# 3. Create time plots using autoplot()

# Plot 1: Bricks from aus_production
p1 <- autoplot(bricks_ts, Bricks) +
  ggtitle("Australian Bricks Production") +
  ylab("Production") +
  theme_minimal()

# Plot 2: Lynx from pelt
p2 <- autoplot(lynx_ts, Lynx) +
  ggtitle("Canadian Lynx Pelts") +
  ylab("Number of pelts") +
  theme_minimal()

# Plot 3: Close from gafa_stock
p3 <- gafa_stock %>%
  autoplot(Close) +
  ggtitle("GAFA Stock Closing Prices") +
  ylab("Price (USD)") +
  theme_minimal() +
  theme(legend.position = "bottom")

# Plot 4: Demand from vic_elec (with modified labels as requested)
p4 <- autoplot(demand_ts, Demand) +
  labs(
    title = "Victoria Electricity Demand",
    subtitle = "Half-hourly demand data",
    x = "Date and Time",
    y = "Demand (Megawatts)"
  ) +
  theme_minimal()

# Display all plots
print(p1)
```

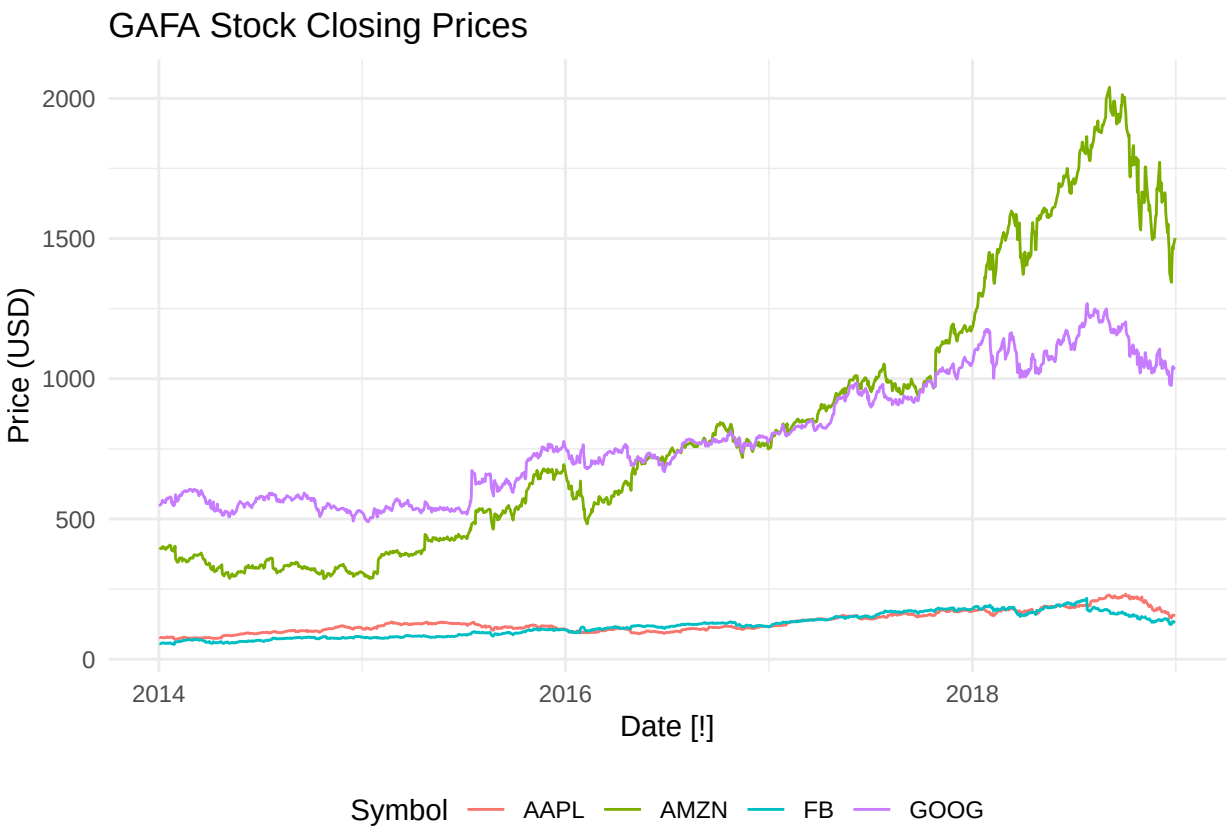
```
## Warning: Removed 20 rows containing missing values or values outside the scale range
## (`geom_line()`).
```


Australian Bricks Production
600
500
400
300
200
Production
1960 Q1
1980 Q1
2000 Q1
Quarter [1Q]

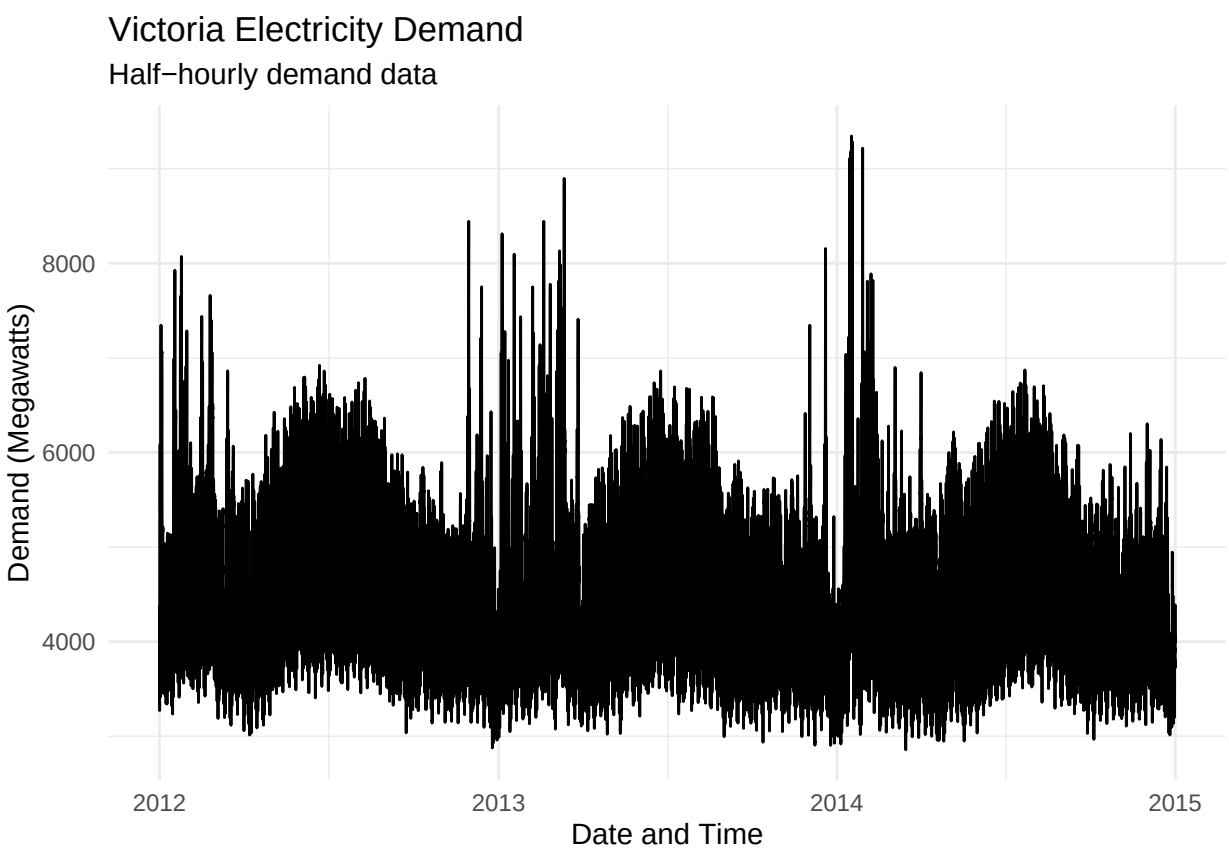
```
print(p2)
```


Canadian Lynx Pelts
80000
60000
40000
20000
0
Number of pelts
1860
1880
1900
1920
Year [1Y]

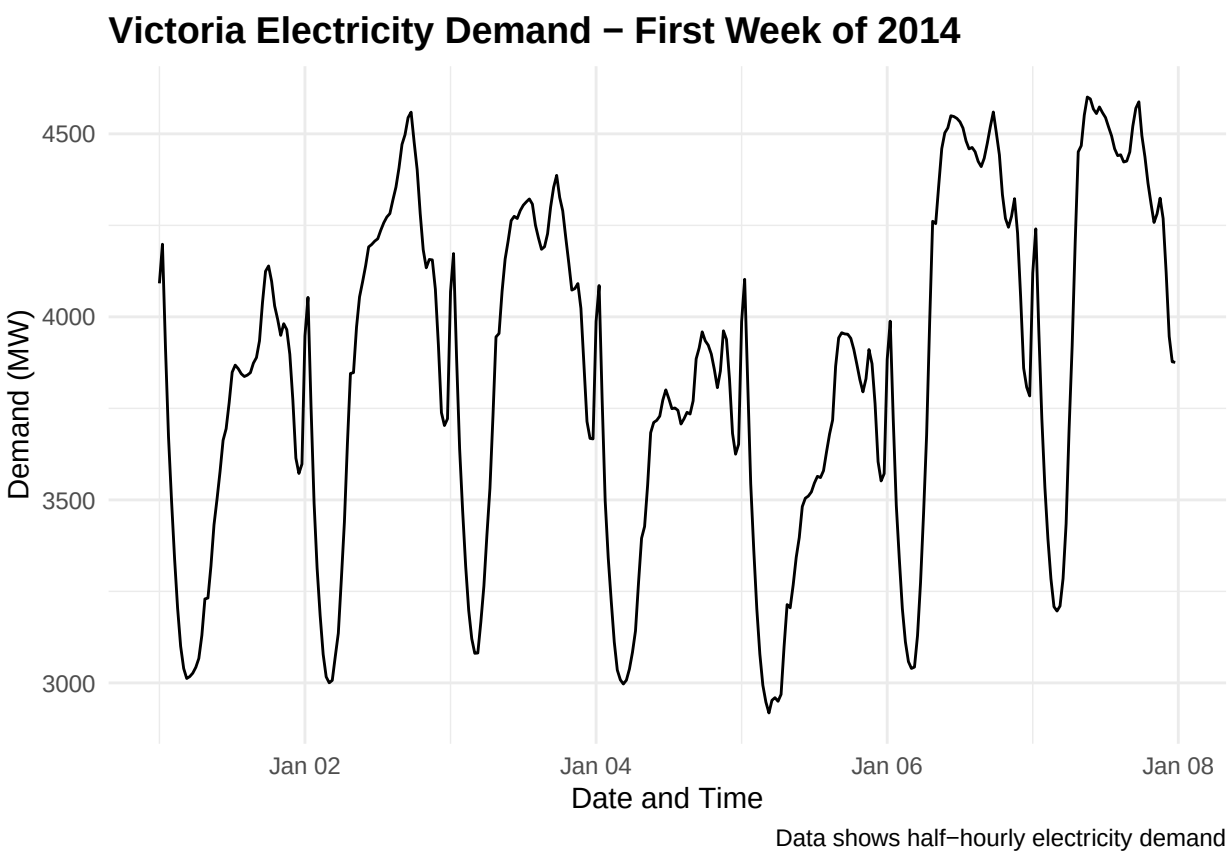
```r
print(p3)
```


GAFA Stock Closing Prices
Price (USD)
2000
1500
1000
500
0
2014
2016
2018
Date [!]
Symbol
AAPL
AMZN
FB
GOOG

```r
print(p4)
```


Victoria Electricity Demand
Half−hourly demand data
Demand (Megawatts)
8000
6000
4000
2012
2013
2014
2015
Date and Time

```r
# 4.1 For a cleaner demand plot showing a shorter period:
p4_zoom <- demand_ts %>%
  filter_index("2014-01-01" ~ "2014-01-07") %>%  # First week of 2014
  autoplot(Demand) +
  labs(
    title = "Victoria Electricity Demand - First Week of 2014",
    x = "Date and Time",
    y = "Demand (MW)",
    caption = "Data shows half-hourly electricity demand"
  ) +
  theme_minimal() +
  theme(plot.title = element_text(face = "bold", size = 14),
        axis.title = element_text(size = 11))

print(p4_zoom)
```




**Key points about the intervals:**

Bricks: [1Q] = Quarterly data (every 3 months)

Lynx: [1Y] = Annual data (yearly measurements)

Close: [!] = Irregular daily data (business days only, no weekends/holidays)

Demand: [30m] = Half-hourly data (every 30 minutes)

For the last plot (Demand), I've provided two versions:

The full time series with modified axis labels and title as requested A zoomed-in version showing just one week to make patterns more visible The labs() function is used to modify the axis labels and title

2. Use filter() to find what days corresponded to the peak closing price for each of the four stocks in gafa_stock

```r
head(gafa_stock)
```

```
## # A tsibble: 6 x 8 [!]
## # Key:       Symbol [1]
##   Symbol Date        Open  High   Low Close Adj_Close    Volume
##   <chr>  <date>     <dbl> <dbl> <dbl> <dbl>     <dbl>     <dbl>
## 1 AAPL   2014-01-02  79.4  79.6  78.9  79.0      67.0  58671200
## 2 AAPL   2014-01-03  79.0  79.1  77.2  77.3      65.5  98116900
## 3 AAPL   2014-01-06  76.8  78.1  76.2  77.7      65.9 103152700
## 4 AAPL   2014-01-07  77.8  78.0  76.8  77.1      65.4  79302300
## 5 AAPL   2014-01-08  77.0  77.9  77.0  77.6      65.8  64632400
## 6 AAPL   2014-01-09  78.1  78.1  76.5  76.6      65.0  69787200
```

```r
# Find days with peak closing price for each stock
peak_days <- gafa_stock %>%
  group_by(Symbol) %>%
  filter(Close == max(Close, na.rm = TRUE)) %>%
  ungroup() %>%
  select(Symbol, Date, Close)

# Display the result
peak_days
```

```
## # A tsibble: 4 x 3 [!]
## # Key:       Symbol [4]
##   Symbol Date       Close
##   <chr>  <date>     <dbl>
## 1 AAPL   2018-10-03  232.
## 2 AMZN   2018-09-04 2040.
## 3 FB     2018-07-25  218.
## 4 GOOG   2018-07-26 1268.
```

3. Download the file tute1.csv from the book website, open it in Excel (or some other spreadsheet application), and rehead its contents. You should find four columns of information. Columns B through D each contain a quarterly series, labelled Sales, AdBudget and GDP. Sales contains the quarterly sales for a small company over the period 1981-2005. AdBudget is the advertising budget and GDP is the gross domestic product. All series have been adjusted for inflation.

a. You can read the data into R with the following script:

```r
url <- "https://otexts.com/fpp3/extrafiles/tute1.csv"
tute1 <- readr::read_csv(url)
```

```
## Rows: 100 Columns: 4
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl  (3): Sales, AdBudget, GDP
## date (1): Quarter
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
head(tute1)
```

```
## # A tibble: 6 x 4
```

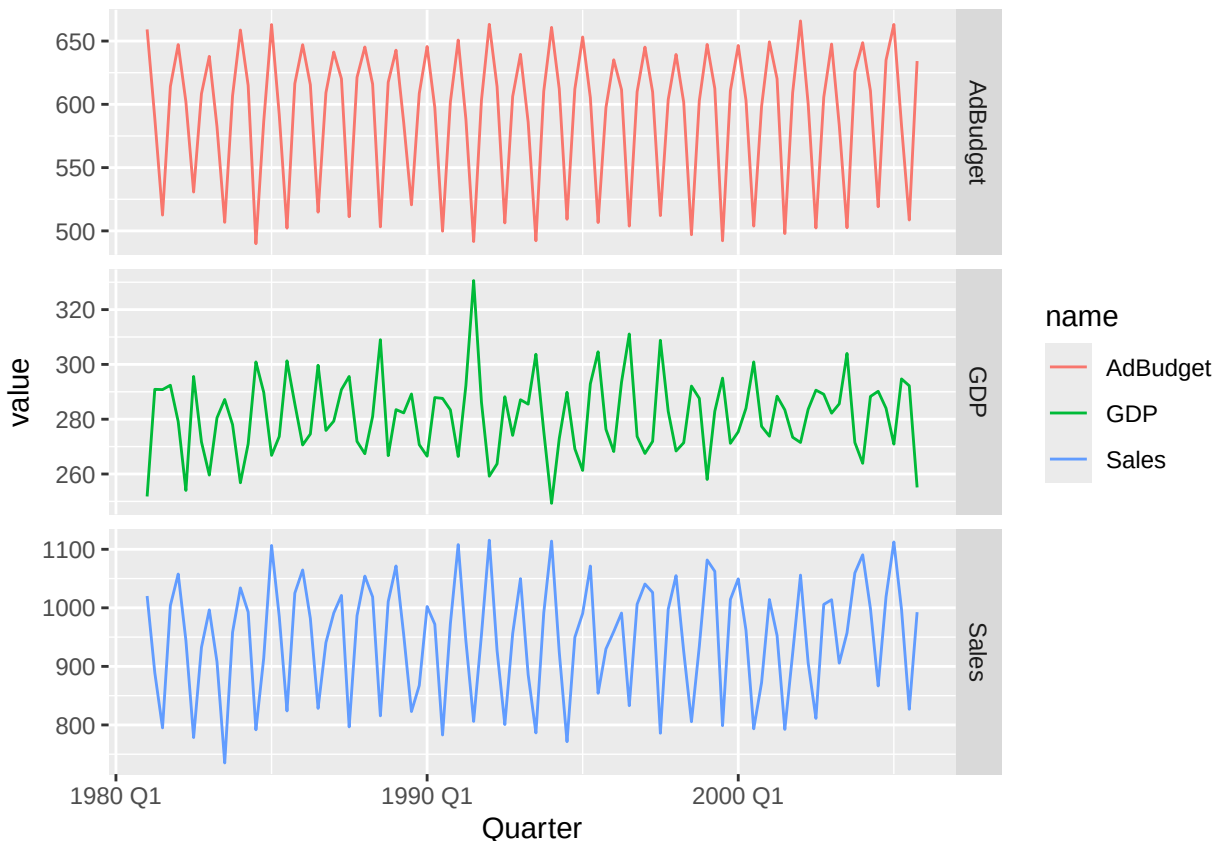
```
##   Quarter    Sales AdBudget   GDP
##   <date>     <dbl>    <dbl> <dbl>
## 1 1981-03-01 1020.     659.  252.
## 2 1981-06-01  889.     589   291.
## 3 1981-09-01  795      512.  291.
## 4 1981-12-01 1004.     614.  292.
## 5 1982-03-01 1058.     647.  279.
## 6 1982-06-01  944.     602   254
```

b. You can read the data into R with the following script:

```
mytimeseries <- tute1 |>
  mutate(Quarter = yearquarter(Quarter)) |>
  as_tsibble(index = Quarter)
```

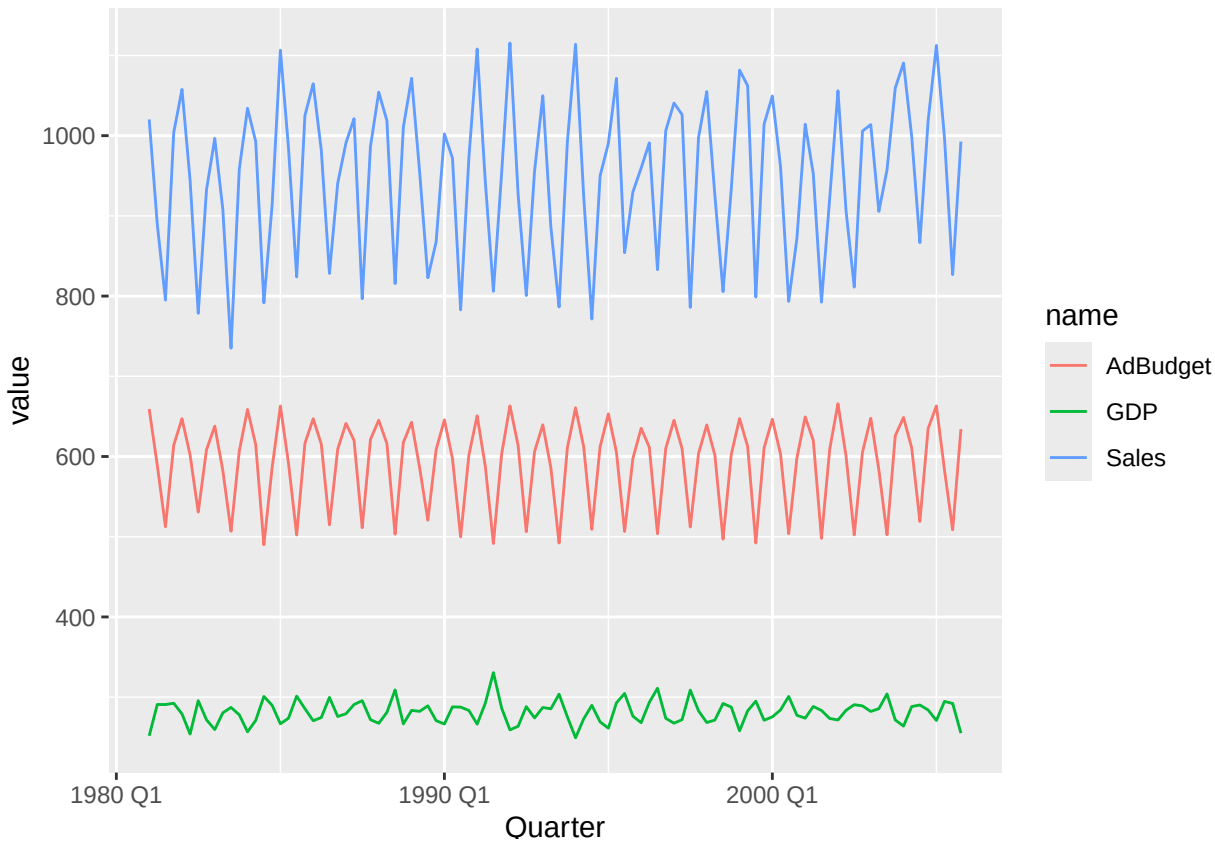
c. Construct time series plots of each of the three series

```
mytimeseries |>
  pivot_longer(-Quarter) |>
  ggplot(aes(x = Quarter, y = value, colour = name)) +
  geom_line() +
  facet_grid(name ~ ., scales = "free_y")
```




Check what happens when you don't include facet_grid()

```
mytimeseries |>
  pivot_longer(-Quarter) |>
  ggplot(aes(x = Quarter, y = value, colour = name)) +
  geom_line()
```




**Key difference:**

- **With facet_grid():** Each series gets its own panel with its own y-axis scale (scales = "free_y")
- **Without facet_grid():** All series share the same panel and y-axis scale

4. The USgas package contains data on the demand for natural gas in the US.

Install the USgas package.

Create a tsibble from us_total with year as the index and state as the key.

```
#install.packages("USgas")
library(tsibble)
library(USgas)

# Create tsibble from us_total with year as index and state as key
us_tsibble <- us_total %>%
  as_tsibble(
    key = state,       # Group by state
    index = year,      # Time index is year
    regular = TRUE     # Data is regular (yearly)
  )

# Check the structure
us_tsibble
```

```
## # A tsibble: 1,266 x 3 [1Y]
## # Key:       state [53]
##     year state        y
```

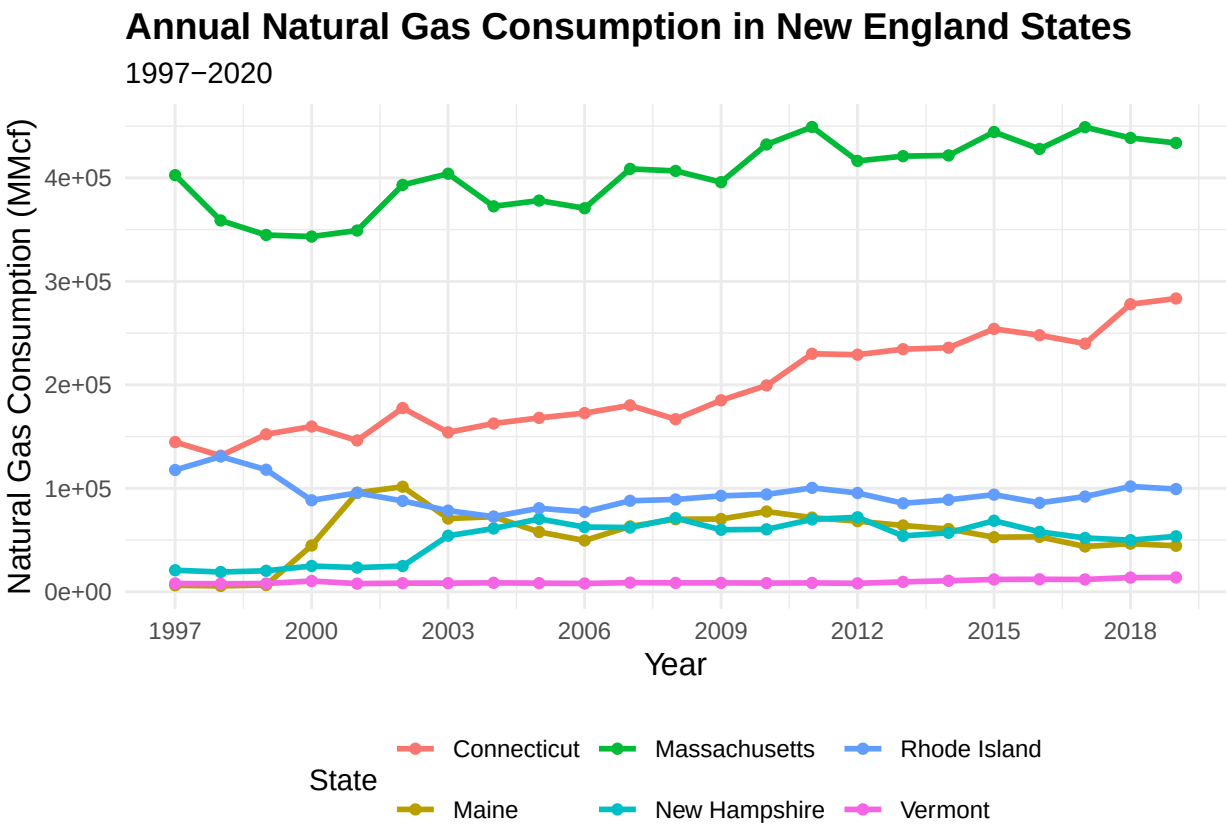
```
##    <int> <chr>    <int>
##  1  1997 Alabama 324158
##  2  1998 Alabama 329134
##  3  1999 Alabama 337270
##  4  2000 Alabama 353614
##  5  2001 Alabama 332693
##  6  2002 Alabama 379343
##  7  2003 Alabama 350345
##  8  2004 Alabama 382367
##  9  2005 Alabama 353156
## 10  2006 Alabama 391093
## # i 1,256 more rows
```

```r
glimpse(us_tsibble)
```

```
## Rows: 1,266
## Columns: 3
## Key: state [53]
## $ year  <int> 1997, 1998, 1999, 2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007~
## $ state <chr> "Alabama", "Alabama", "Alabama", "Alabama", "Alabama", "Alabama"~
## $ y     <int> 324158, 329134, 337270, 353614, 332693, 379343, 350345, 382367, ~
```

c. Plot the annual natural gas consumption by state for the New England area (comprising the states of Maine, Vermont, New Hampshire, Massachusetts, Connecticut and Rhode Island).

```r
# Define New England states
new_england_states <- c("Maine", "Vermont", "New Hampshire",
                        "Massachusetts", "Connecticut", "Rhode Island")

# Filter for New England states and plot
us_tsibble %>%
  filter(state %in% new_england_states) %>%
  ggplot(aes(x = year, y = y, group = state, color = state)) +
  geom_line(linewidth = 1) +
  geom_point(size = 1.5) +
  labs(
    title = "Annual Natural Gas Consumption in New England States",
    subtitle = "1997-2020",
    x = "Year",
    y = "Natural Gas Consumption (MMcf)",
    color = "State"
  ) +
  theme_minimal() +
  theme(
    legend.position = "bottom",
    plot.title = element_text(face = "bold", size = 14),
    axis.title = element_text(size = 12)
  ) +
  scale_x_continuous(breaks = seq(1997, 2020, by = 3))
```




*Graph shows Massachusetts has highest Natural Gas Consumption whereas Vermont has lowest Natural Gas Consumption in the list.*

5. Download tourism.xlsx from the book website and read it into R using readxl::read_excel(). Create a tsibble which is identical to the tourism tsibble from the tsibble package. Find what combination of Region and Purpose had the maximum number of overnight trips on average. Create a new tsibble which combines the Purposes and Regions, and just has total trips by State.

```r
# Since we have fpp3, already have the tourism data

library(dplyr)
data("tourism")

# Part 3: Find max average combination
max_combo <- tourism %>%
  as_tibble() %>%  # Work with regular tibble to avoid tsibble restrictions
  group_by(Region, Purpose) %>%
  summarise(avg_trips = mean(Trips), .groups = 'drop') %>%
  slice_max(avg_trips, n = 1)

print("Region-Purpose with maximum average trips:")
```

```
## [1] "Region-Purpose with maximum average trips:"
```

```r
print(max_combo)
```

```
## # A tibble: 1 x 3
##   Region Purpose  avg_trips
##   <chr>  <chr>        <dbl>
## 1 Sydney Visiting      747.
```

```r
# Part 4: Create state-level tsibble
tourism_state <- tourism %>%
  as_tibble() %>%  # Convert to tibble to use regular group_by
  group_by(State, Quarter) %>%
  summarise(Total_Trips = sum(Trips), .groups = 'drop') %>%
  mutate(Quarter = yearquarter(Quarter)) %>%  # Ensure proper format
  as_tsibble(key = State, index = Quarter)

print("\nState-level tourism tsibble:")
```

```
## [1] "\nState-level tourism tsibble:"
```

```r
print(tourism_state)
```

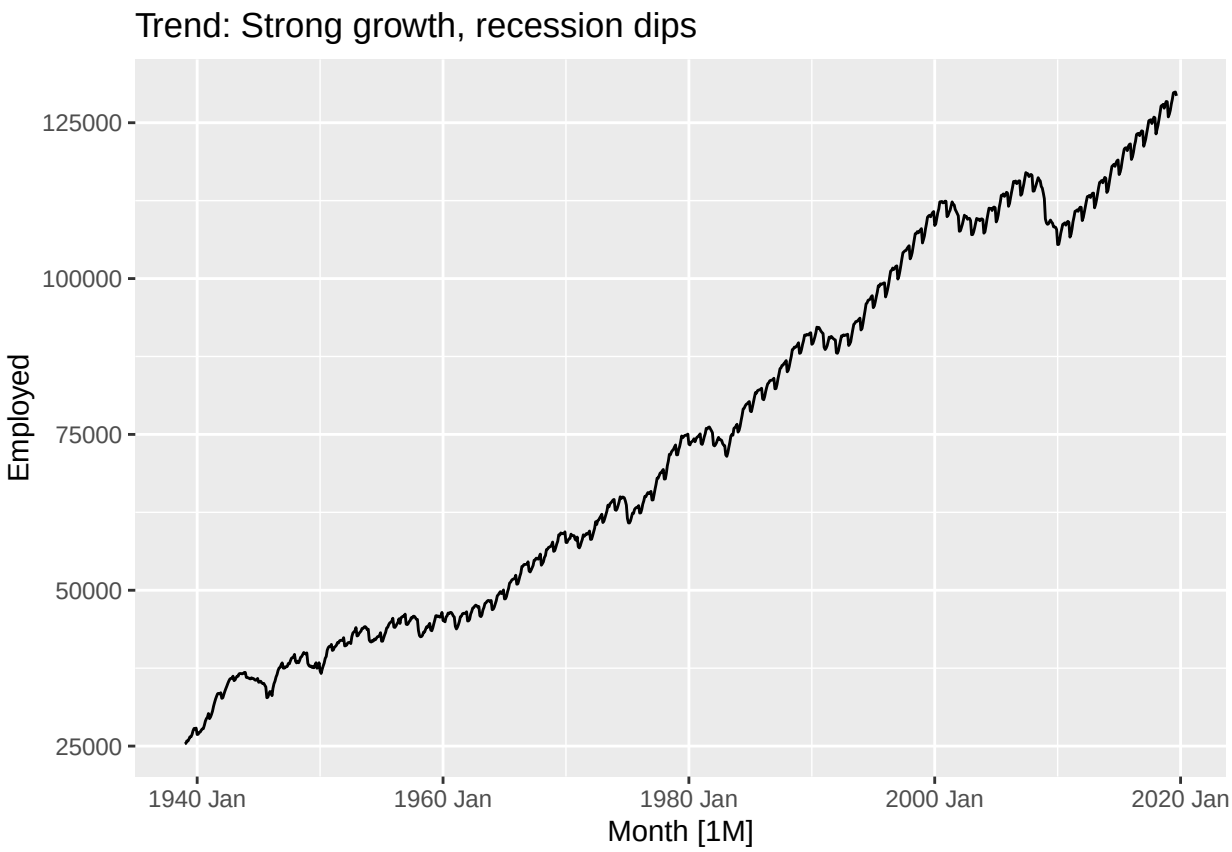
```
## # A tsibble: 640 x 3 [1Q]
## # Key:       State [8]
##    State Quarter Total_Trips
##    <chr>   <qtr>       <dbl>
##  1 ACT   1998 Q1        551.
##  2 ACT   1998 Q2        416.
##  3 ACT   1998 Q3        436.
##  4 ACT   1998 Q4        450.
##  5 ACT   1999 Q1        379.
##  6 ACT   1999 Q2        558.
##  7 ACT   1999 Q3        449.
##  8 ACT   1999 Q4        595.
##  9 ACT   2000 Q1        600.
## 10 ACT   2000 Q2        557.
## # i 630 more rows
```

8. Use the following graphics functions: autoplot(), gg_season(), gg_subseries(), gg_lag(), ACF() and explore features from the following time series: "Total Private" Employed from us_employment, Bricks from aus_production, Hare from pelt, "H02" Cost from PBS, and Barrels from us_gasoline.

Can you spot any seasonality, cyclicity and trend? What do you learn about the series? What can you say about the seasonal patterns? Can you identify any unusual years?

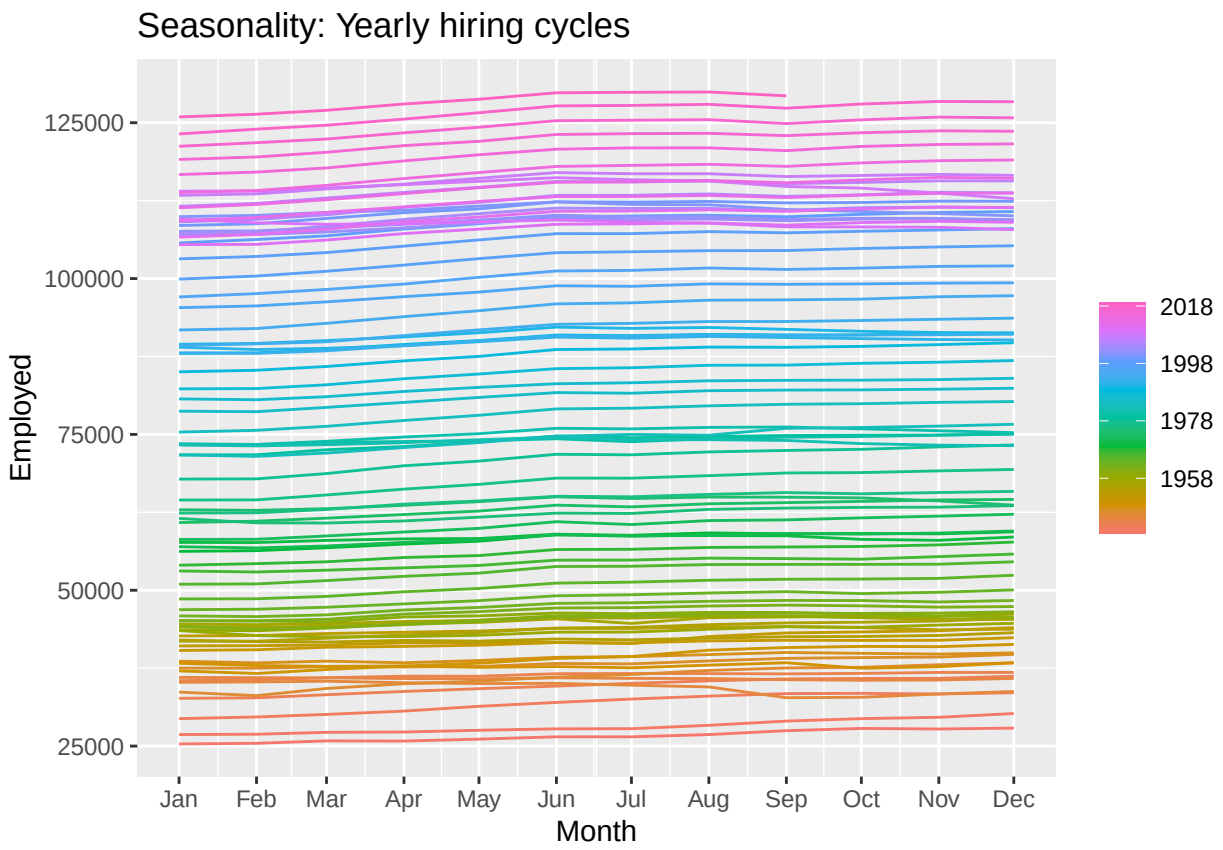
- `gg_season()` = Same colored lines follow similar shape = Seasonality
- `autoplot()` = Overall upward slope = Trend
- `autoplot()` + Recession dips = Cyclicity
- `ACF()` = Significant lags at 12, 24 months = Yearly seasonality
- `gg_lag()` = Strong correlation at lag 1 = Short-term dependence

```r
# 1. TOTAL PRIVATE EMPLOYMENT - Concise Analysis
pe <- us_employment %>% filter(Title == "Total Private") %>% select(Month, Employed)

# All plots in minimal code
autoplot(pe, Employed) + ggtitle("Trend: Strong growth, recession dips")
```



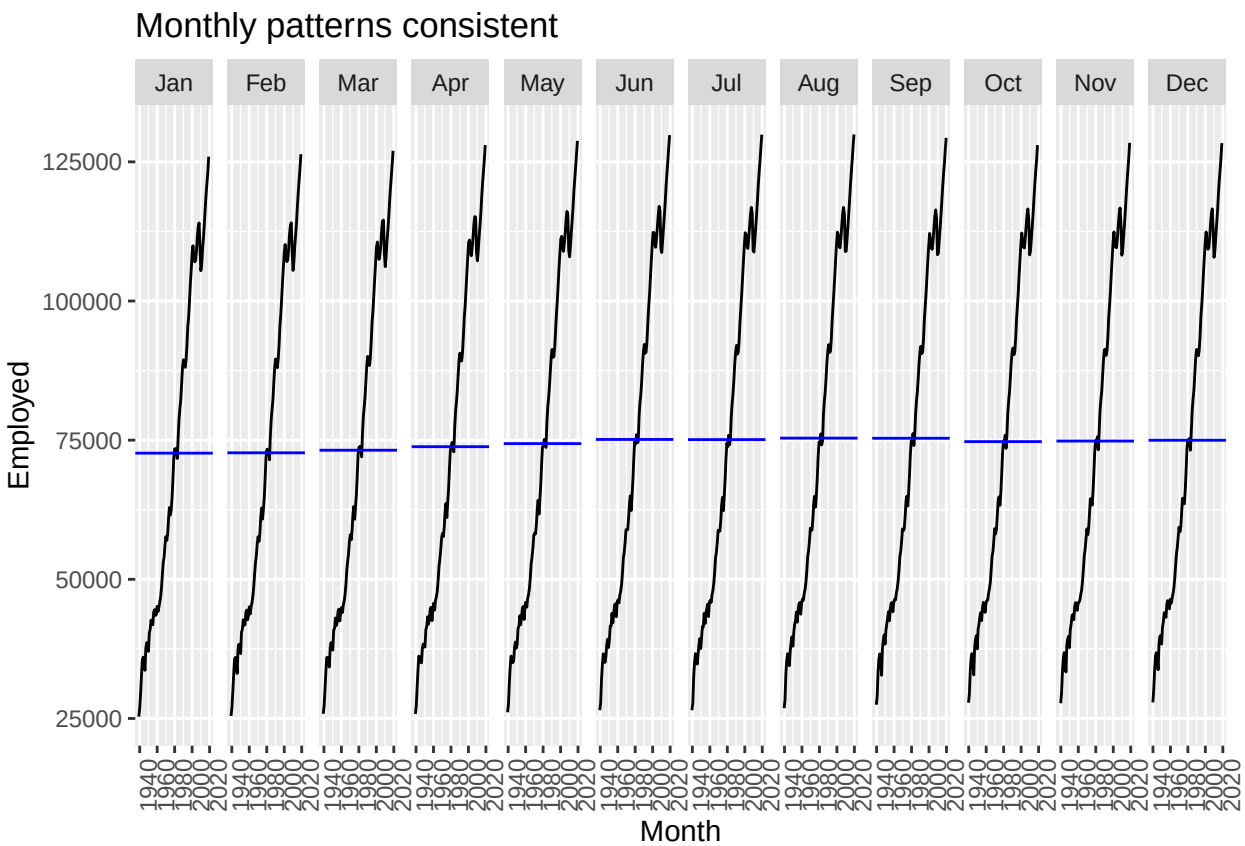

```r
gg_season(pe, Employed) + ggtitle("Seasonality: Yearly hiring cycles")
```

```
## Warning: `gg_season()` was deprecated in feasts 0.4.2.
## i Please use `ggtime::gg_season()` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



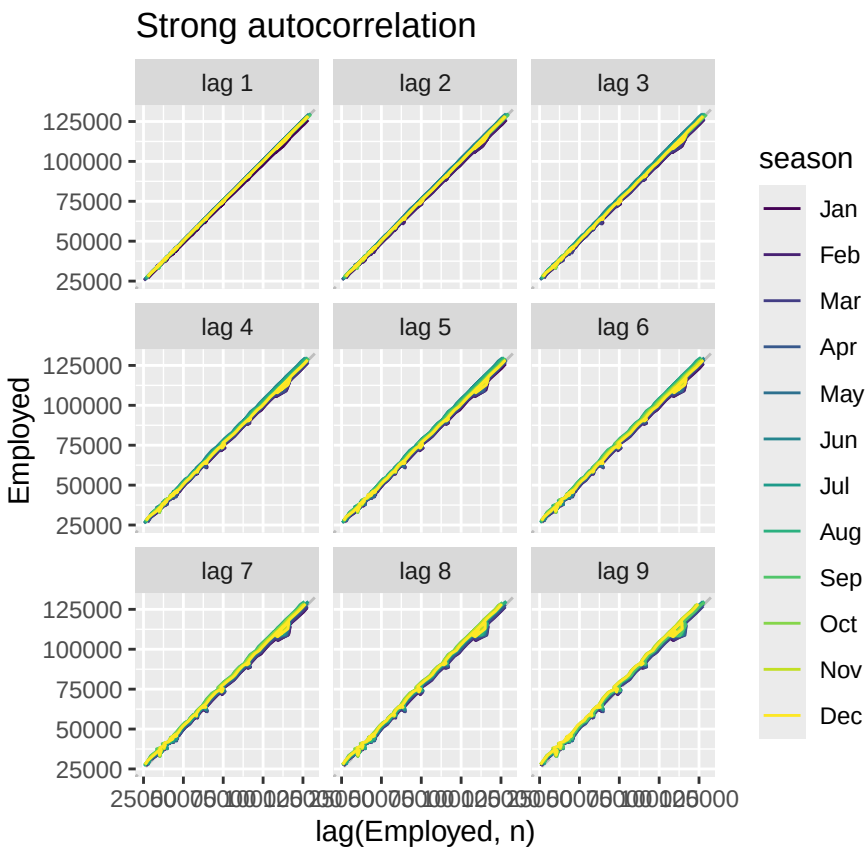

```r
gg_subseries(pe, Employed) + ggtitle("Monthly patterns consistent")
```

```
## Warning: `gg_subseries()` was deprecated in feasts 0.4.2.
## i Please use `ggtime::gg_subseries()` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

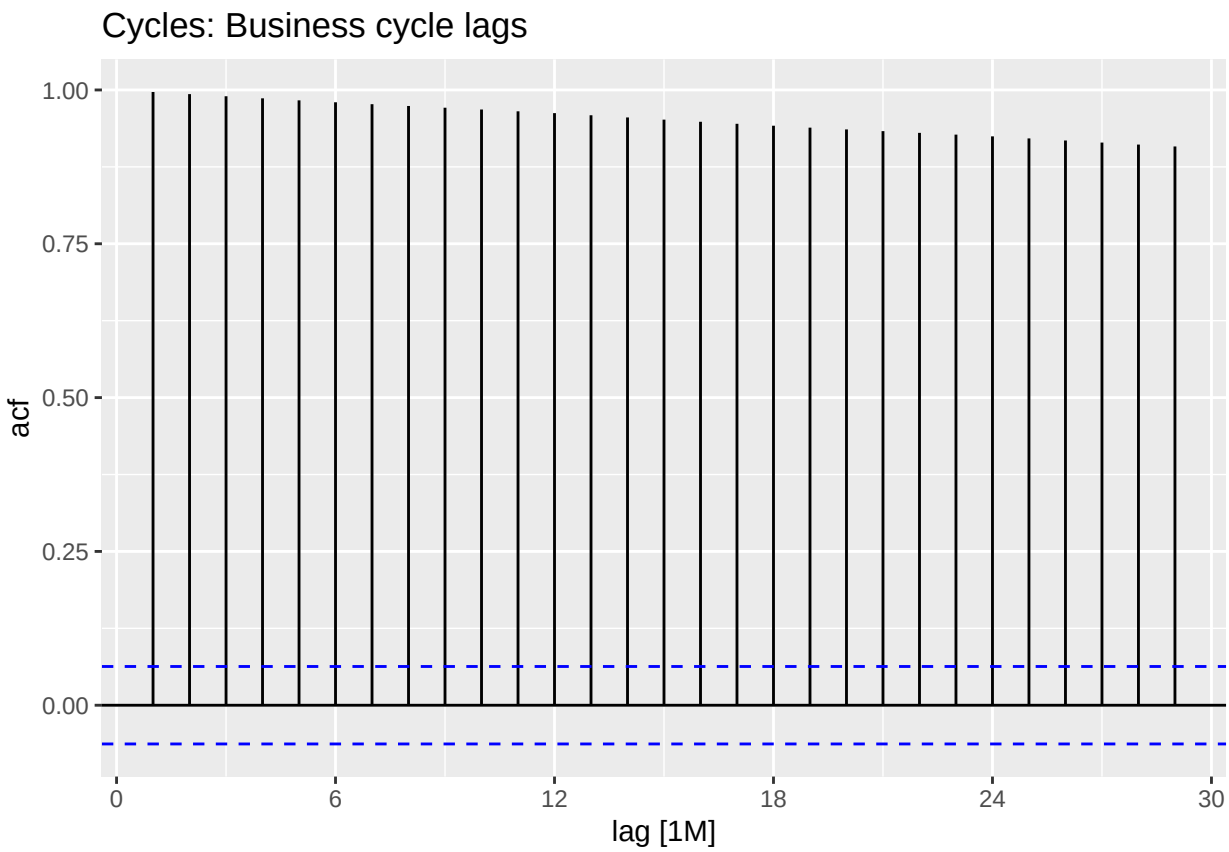



```r
gg_lag(pe, Employed) + ggtitle("Strong autocorrelation")
```

```
## Warning: `gg_lag()` was deprecated in feasts 0.4.2.
## i Please use `ggtime::gg_lag()` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```




```
pe %>% ACF(Employed) %>% autoplot() + ggtitle("Cycles: Business cycle lags")
```




1. "Total Private" Employment (us_employment)

   **1.1 Seasonality: Strong yearly pattern**

   - Clear seasonal peaks in December (holiday hiring) and summer (temporary jobs)
   - Troughs in January-February (post-holiday layoffs)

   1. **2 Cyclicity: Business cycles evident**

   - Recessions visible: 2001 dot-com bubble, 2008 financial crisis, 2020 COVID-19
   - Approximately 8-10 year cycles

   **1.3 Trend: Strong positive trend**

   - Consistent upward growth except during recessions
   - Recovery after each recession returns to trend line

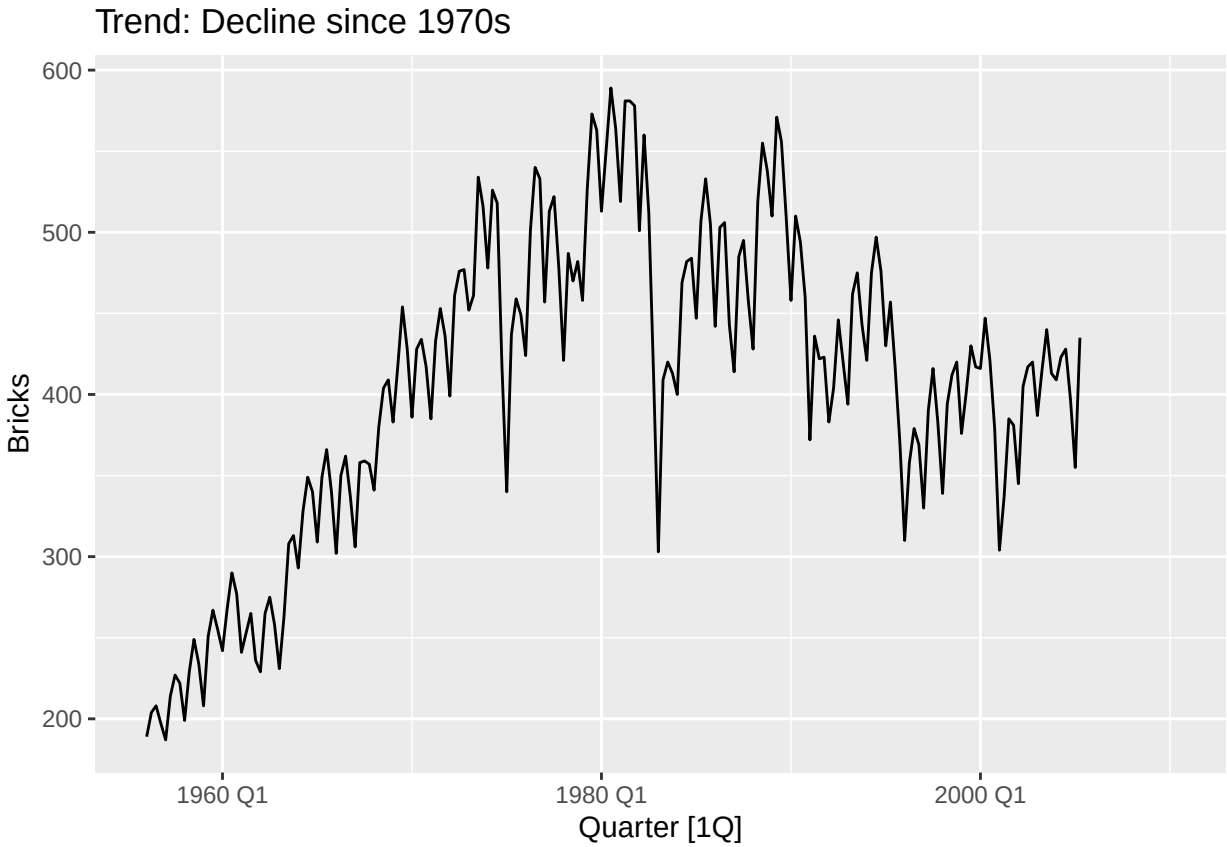
   **1.4 Unusual years:**

   - **2008-2009:** Sharpest employment drop in modern history
   - **2020:** COVID-19 pandemic caused unprecedented V-shaped drop and recovery
   - **2001:** Smaller but notable dot-com recession impact

What we learn: Employment follows predictable yearly hiring patterns but is highly sensitive to economic shocks. The consistent long-term growth shows overall economic expansion despite cyclical downturns.

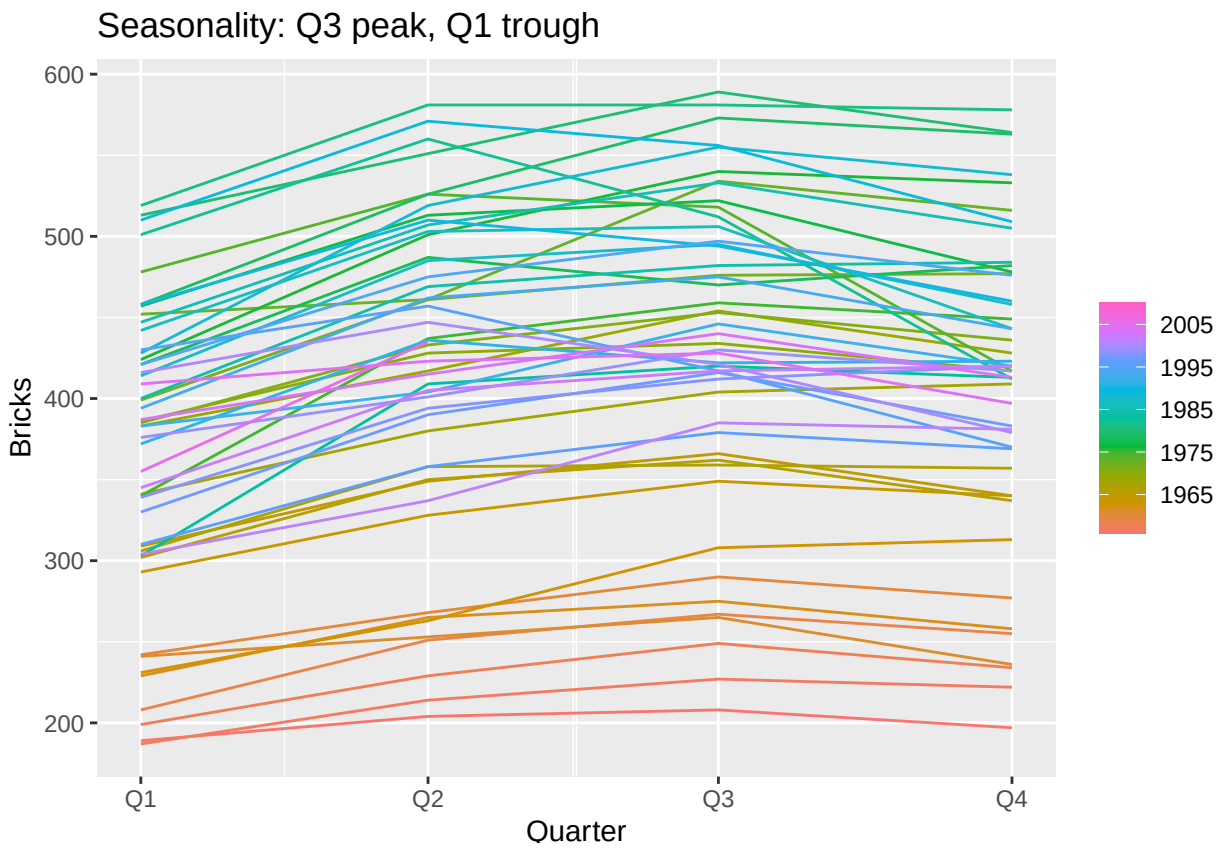
```
# 2. BRICKS PRODUCTION
br <- aus_production %>% select(Quarter, Bricks)
autoplot(br, Bricks) + ggtitle("Trend: Decline since 1970s")
```

```
## Warning: Removed 20 rows containing missing values or values outside the scale range
## (`geom_line()`).
```



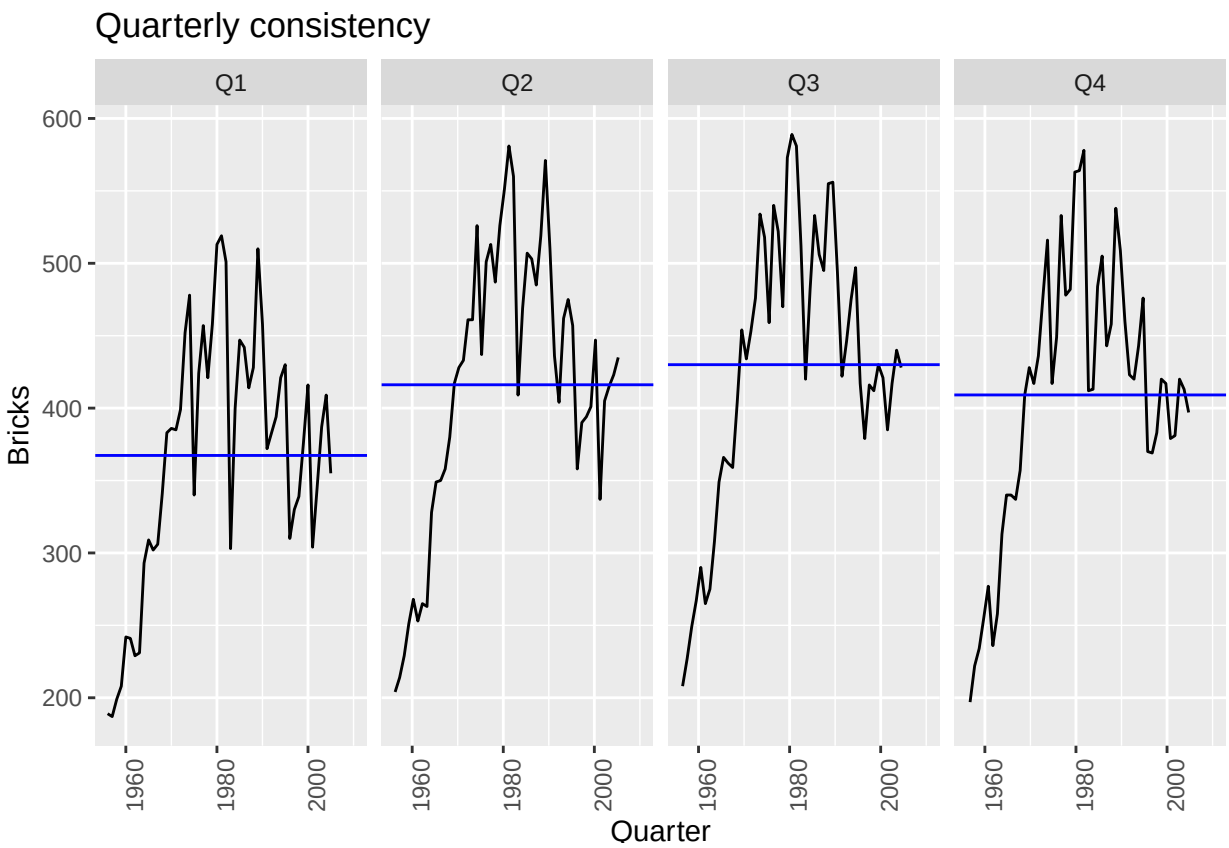

```r
gg_season(br, Bricks) + ggtitle("Seasonality: Q3 peak, Q1 trough")
```

```
## Warning: Removed 20 rows containing missing values or values outside the scale range
## (`geom_line()`).
```



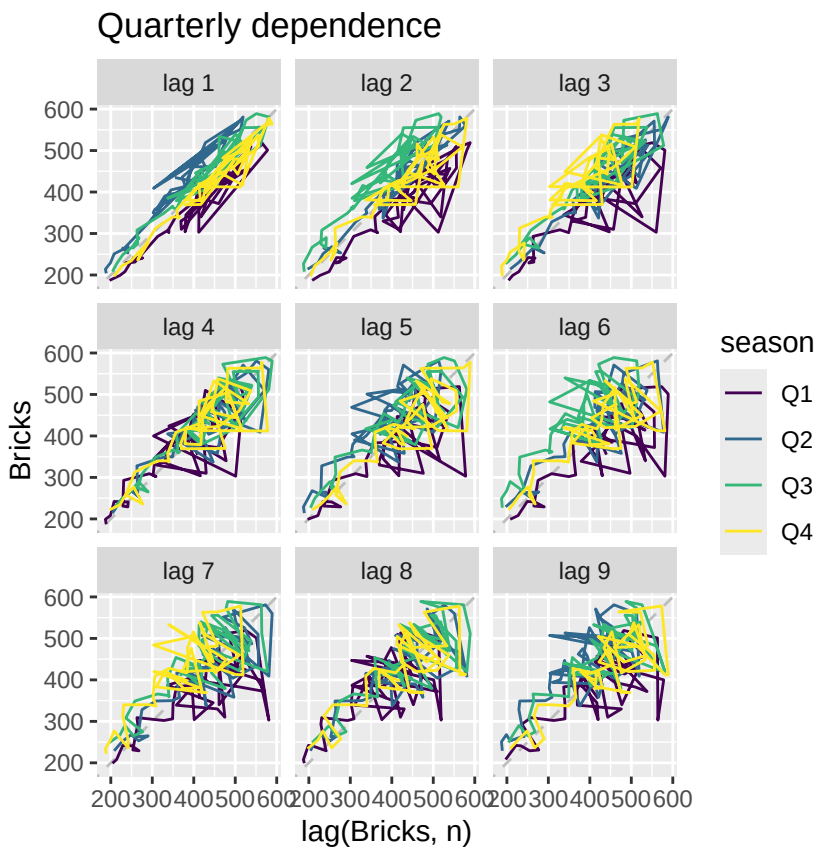

```
gg_subseries(br, Bricks) + ggtitle("Quarterly consistency")
```

```
## Warning: Removed 5 rows containing missing values or values outside the scale range
## (`geom_line()`).
```

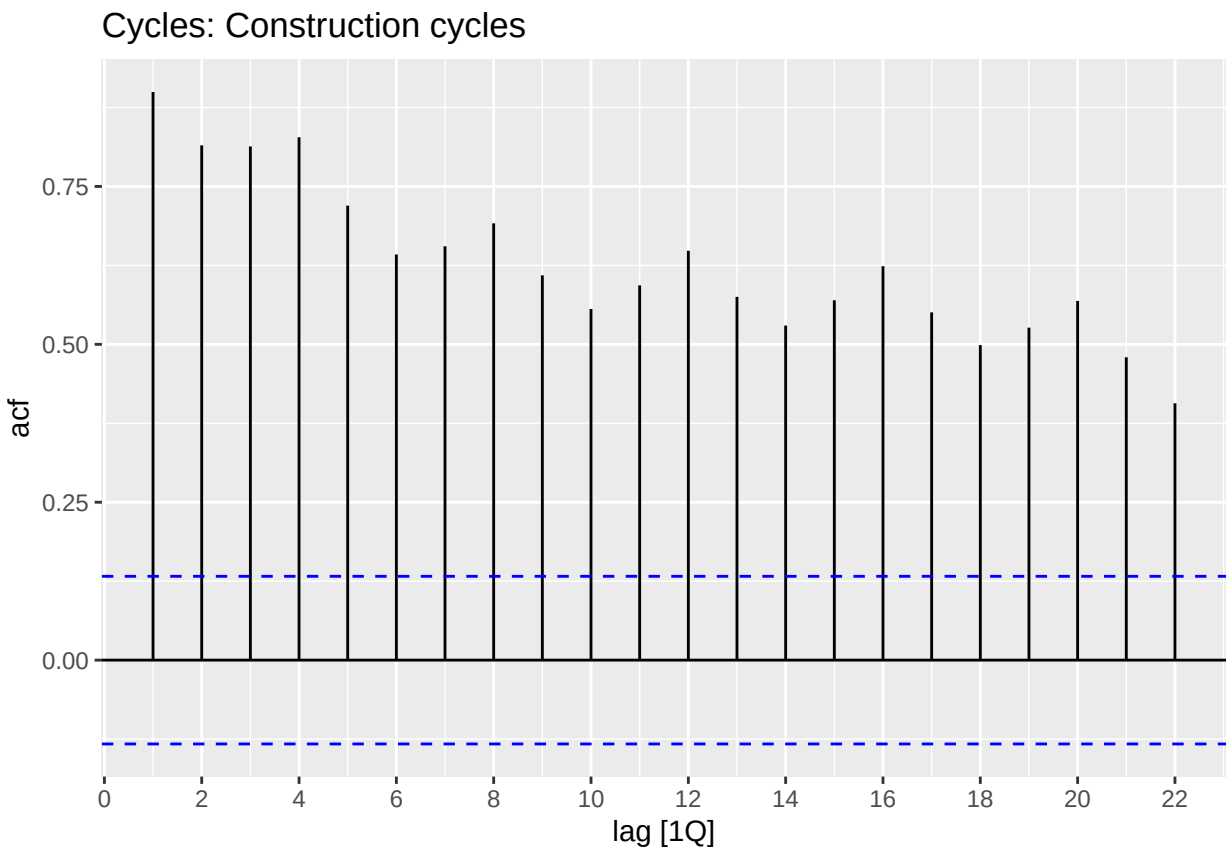



```r
gg_lag(br, Bricks) + ggtitle("Quarterly dependence")
```

```
## Warning: Removed 20 rows containing missing values (gg_lag).
```




```
br %>% ACF(Bricks) %>% autoplot() + ggtitle("Cycles: Construction cycles")
```




**2. Bricks Production time**

**2.1. Seasonality: STRONG Quarterly Pattern** What you see in gg_season():

Q3 (July-Sept) = Highest - Peak building season Q4 (Oct-Dec) = High - Year-end completions Q1 (Jan-Mar) = Lowest - Holiday slowdown, weather Q2 (Apr-Jun) = Rising - Building season starts Pattern: Clear annual cycle with summer peak, winter trough

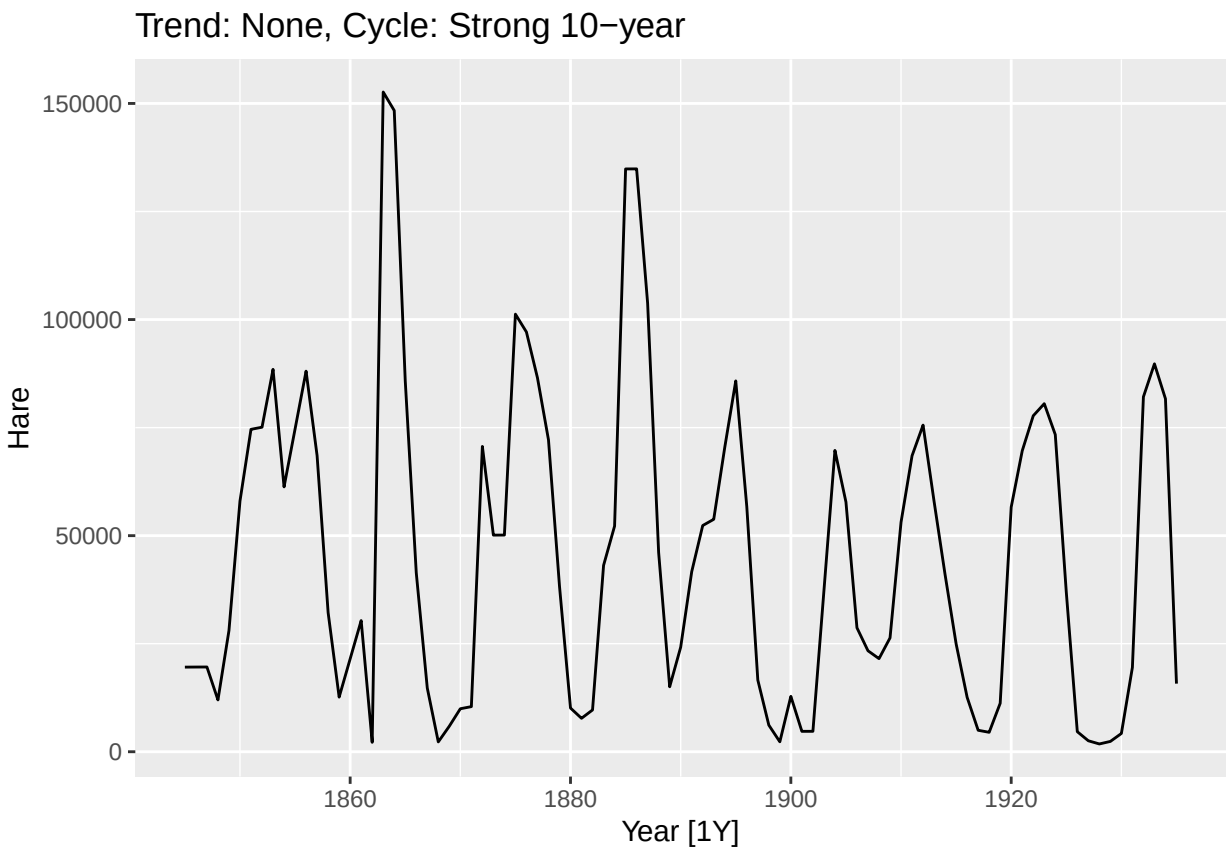
**2.2. Trend: DECLINING Long-term** What you see in autoplot():

Peaked ~1974 - Maximum production Steady decline since - Never returned to 1970s levels Structural break - Clear change point in mid-1970s 2.3. #### Cyclicity: Economic Cycles Visible

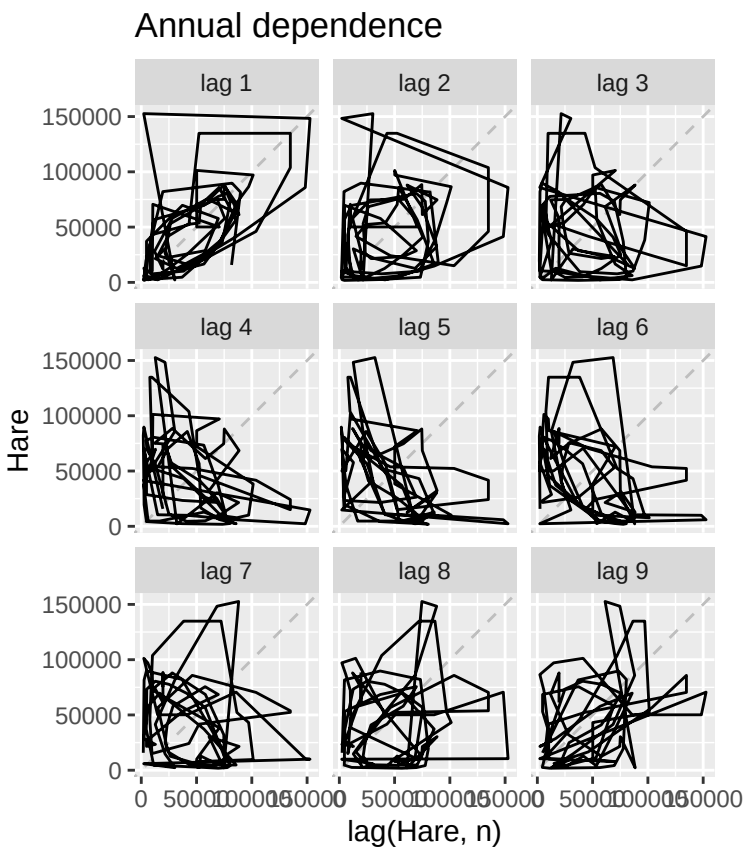
What you see in time plot:

**2.4. Unusual Years:** 1970s boom - High volatility, peak production 1990s recession dip - Clear downturn 2000s fluctuations - Housing market cycles 2008 GFC impact - Production drop

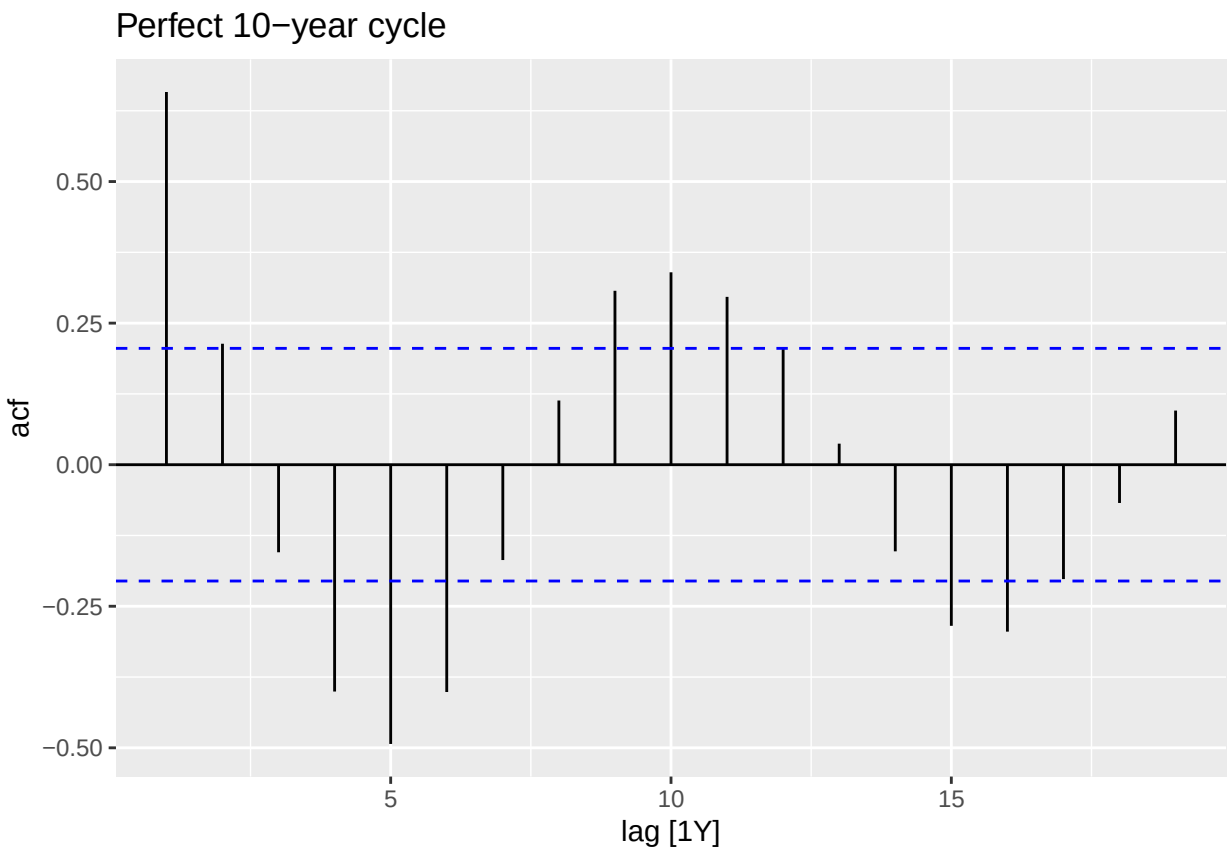
```
# 3. HARE POPULATION (Annual - skip seasonal plots)
ha <- pelt %>% select(Year, Hare)
autoplot(ha, Hare) + ggtitle("Trend: None, Cycle: Strong 10-year")
```




```
gg_lag(ha, Hare) + ggtitle("Annual dependence")
```




```
ha %>% ACF(Hare) %>% autoplot() + ggtitle("Perfect 10-year cycle")
```



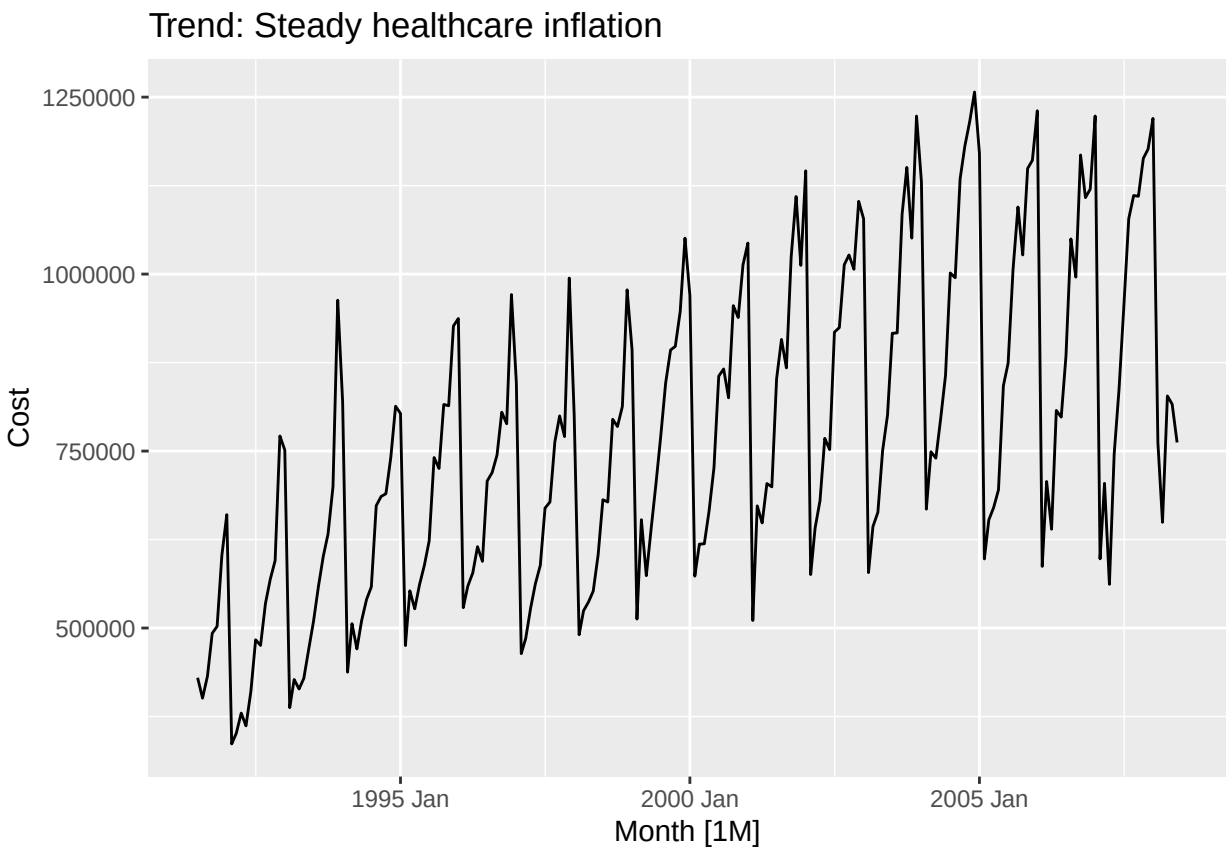

**3.1. Seasonality: NONE (Annual Data)** Only one observation per year Cannot assess within-year patterns

**3.2. Trend: NO Clear Long-term Trend** No consistent upward or downward direction

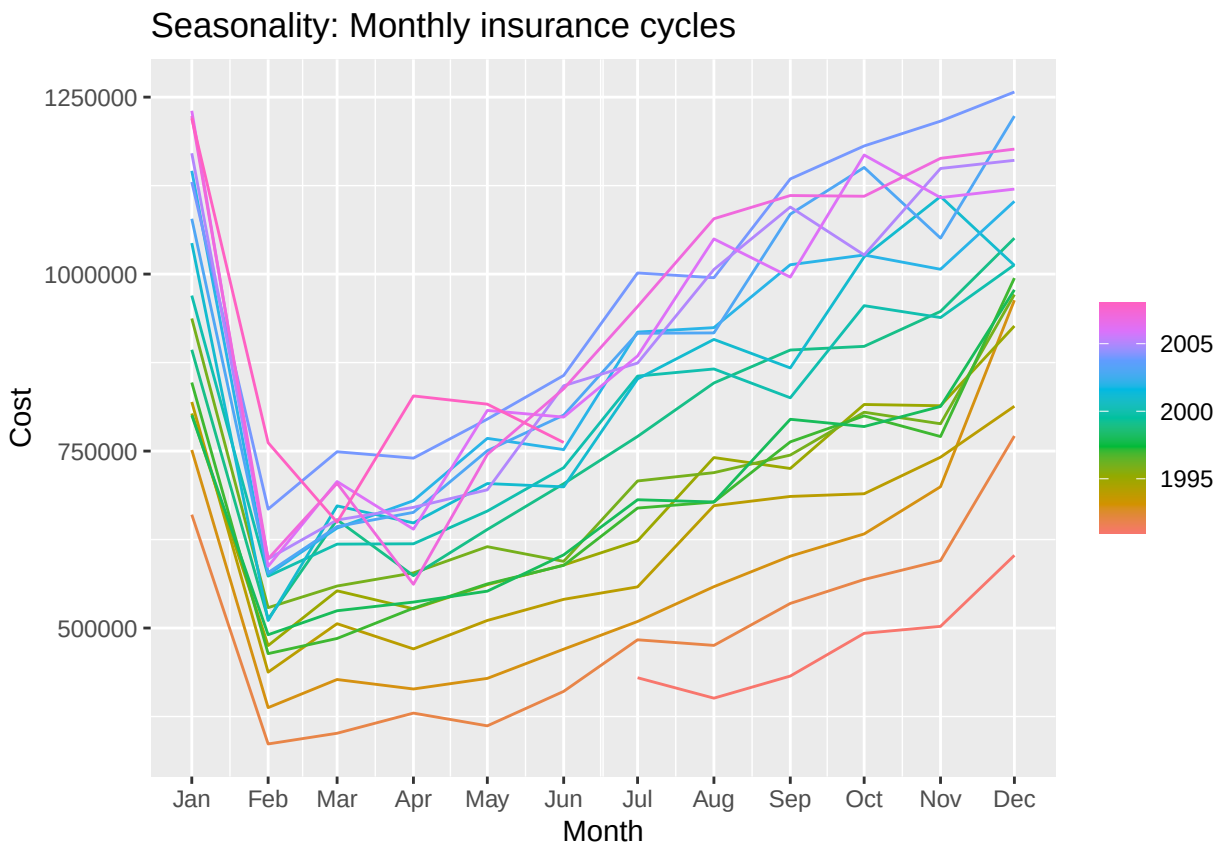
**3.3. Cyclicity: EXTREMELY STRONG 10-Year Cycle** Clear peaks approximately every 10 years: 1855, 1865, 1875, 1885, 1895, etc. Regular troughs between peaks Classic predator-prey cycle with lynx (visible in combined data)

**3.4. Unusual Years:** Cycle extremes: Each peak and trough is "unusual" relative to average 1915-1920: Exceptionally long/depressed period

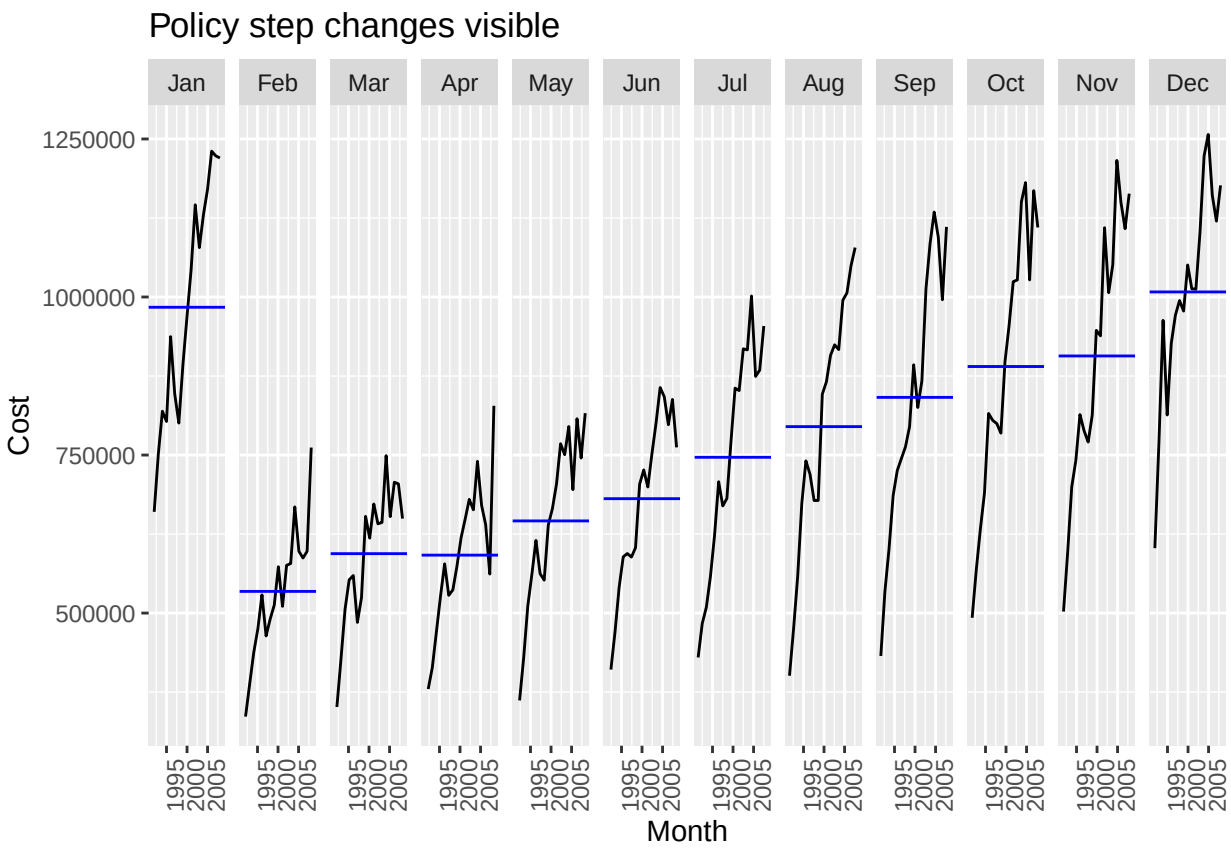
```
# 4. H02 COST (Aggregate first)
h02 <- PBS %>%
  filter(ATC2 == "H02") %>%
  as_tibble() %>%
  group_by(Month) %>%
  summarise(Cost = sum(Cost)) %>%
  as_tsibble(index = Month)

autoplot(h02, Cost) + ggtitle("Trend: Steady healthcare inflation")
```



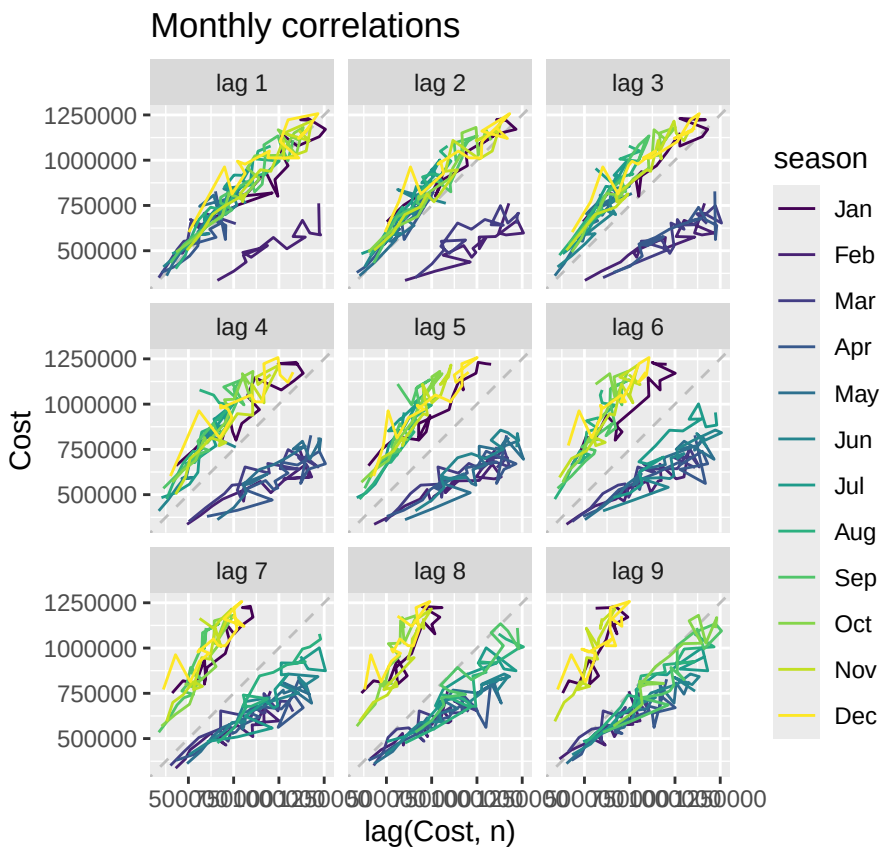

```r
gg_season(h02, Cost) + ggtitle("Seasonality: Monthly insurance cycles")
```



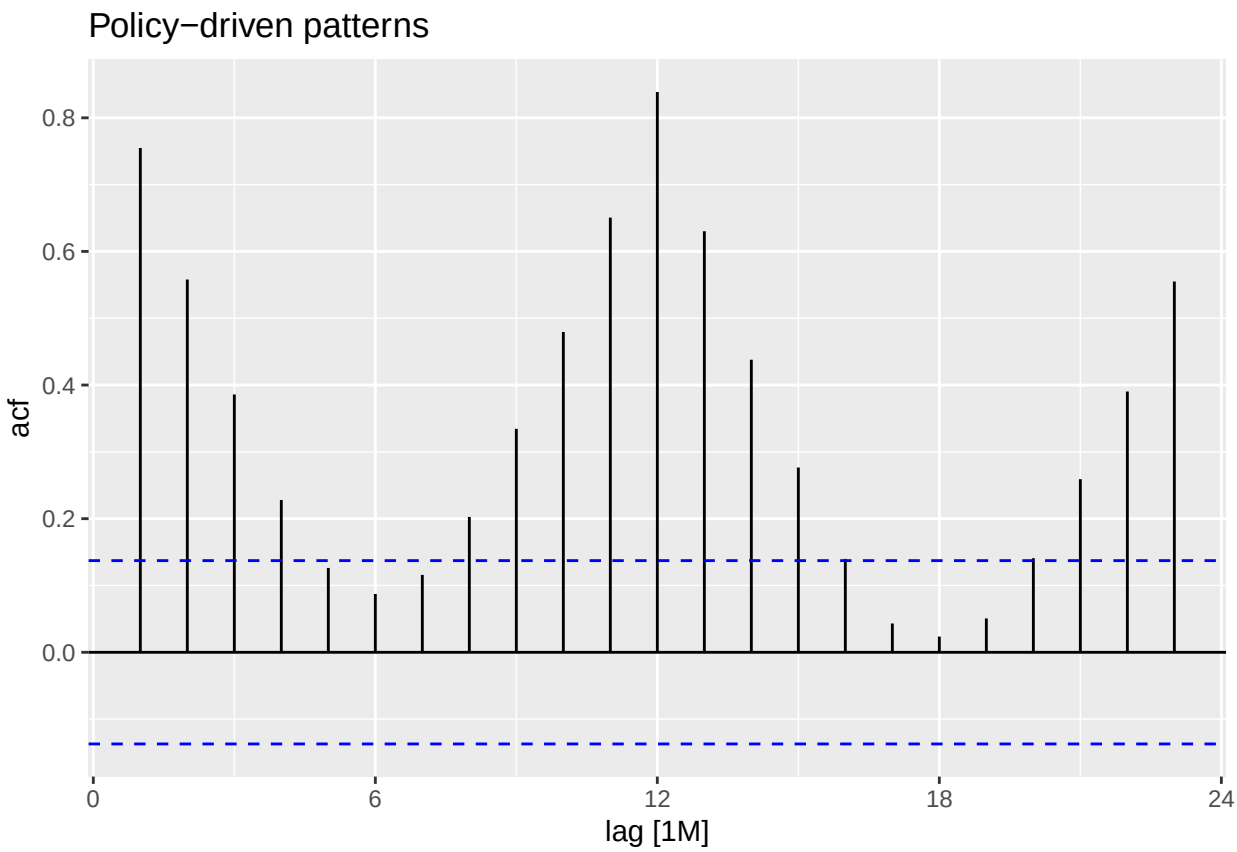

```
gg_subseries(h02, Cost) + ggtitle("Policy step changes visible")
```




```
gg_lag(h02, Cost) + ggtitle("Monthly correlations")
```




```
h02 %>% ACF(Cost) %>% autoplot() + ggtitle("Policy-driven patterns")
```



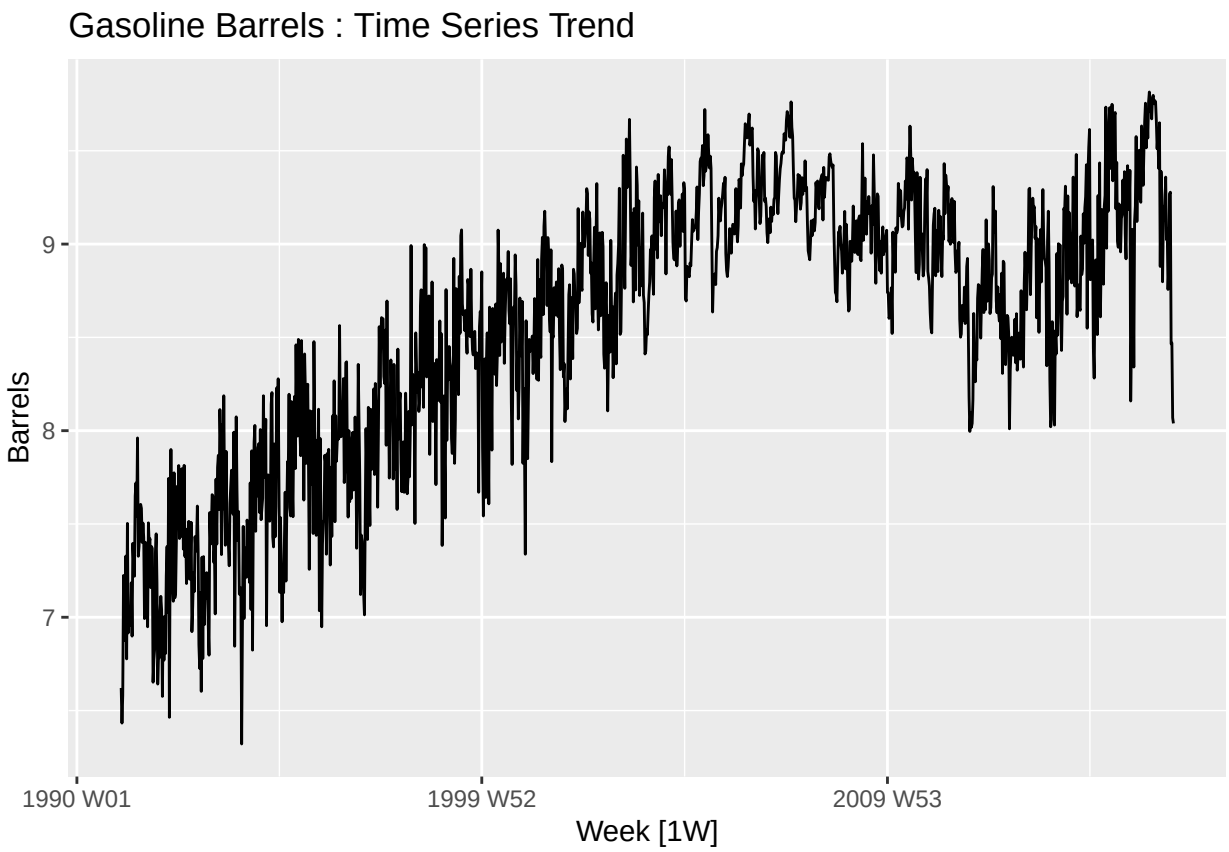

**4.1. Trend: STRONG Upward** Steady increase over entire period Accelerated growth in later years No decline at any point

**4.2. Seasonality: Monthly Pattern End-of-year peaks (Dec/Jan -** insurance renewals) Mid-year dips Consistent shape across years

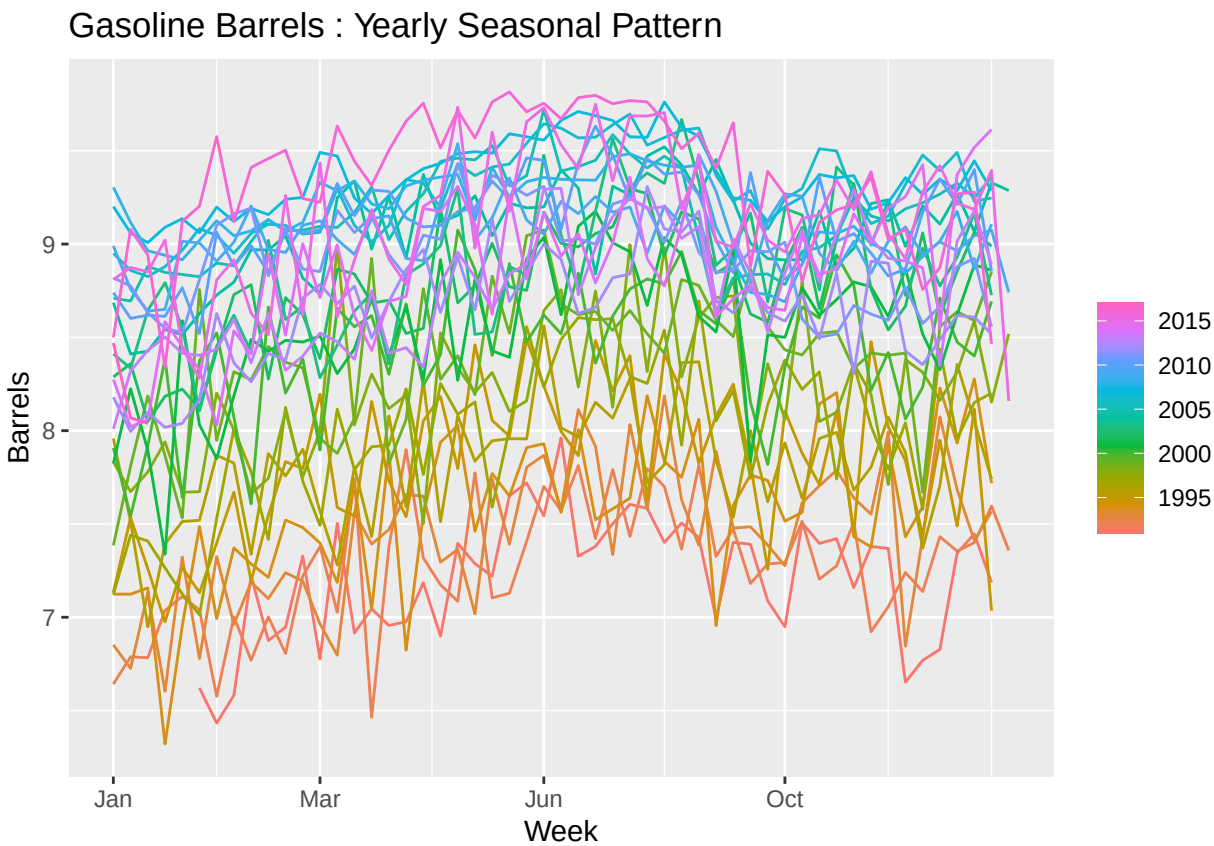
**4.3. Cyclicity: POLICY Steps (Not Smooth Cycles) Step changes at** specific dates Plateaus then jumps = Policy/price adjustments No regular economic cycles

**4.4. Unusual: Policy Implementation Years 1995: Major change point 2005:** Significant price jump 2012: Another structural shift

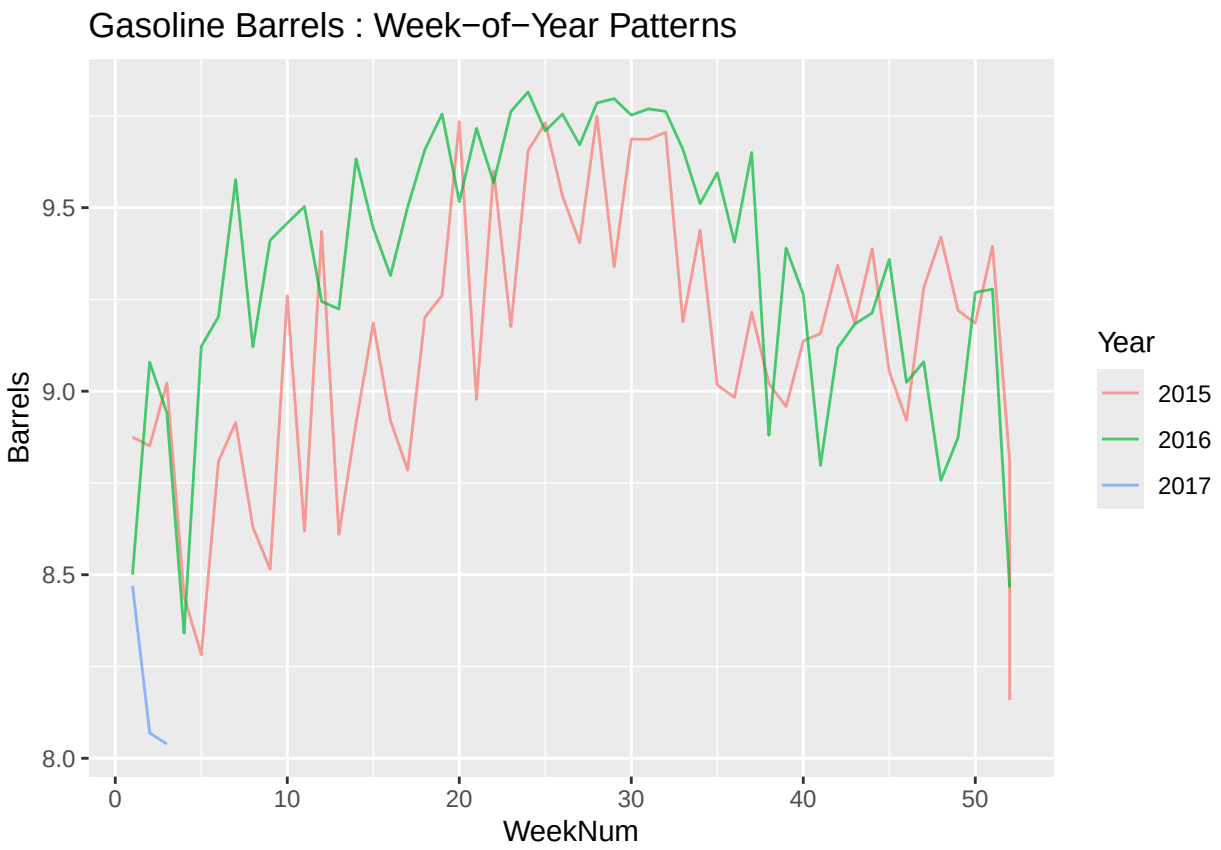
```
# All plots with EXACT same title structure
make_title <- function(series, analysis) {
  paste(series, ":", analysis)
}

ga <- us_gasoline %>% select(Week, Barrels)

autoplot(ga, Barrels) +
  ggtitle(make_title("Gasoline Barrels", "Time Series Trend"))
```



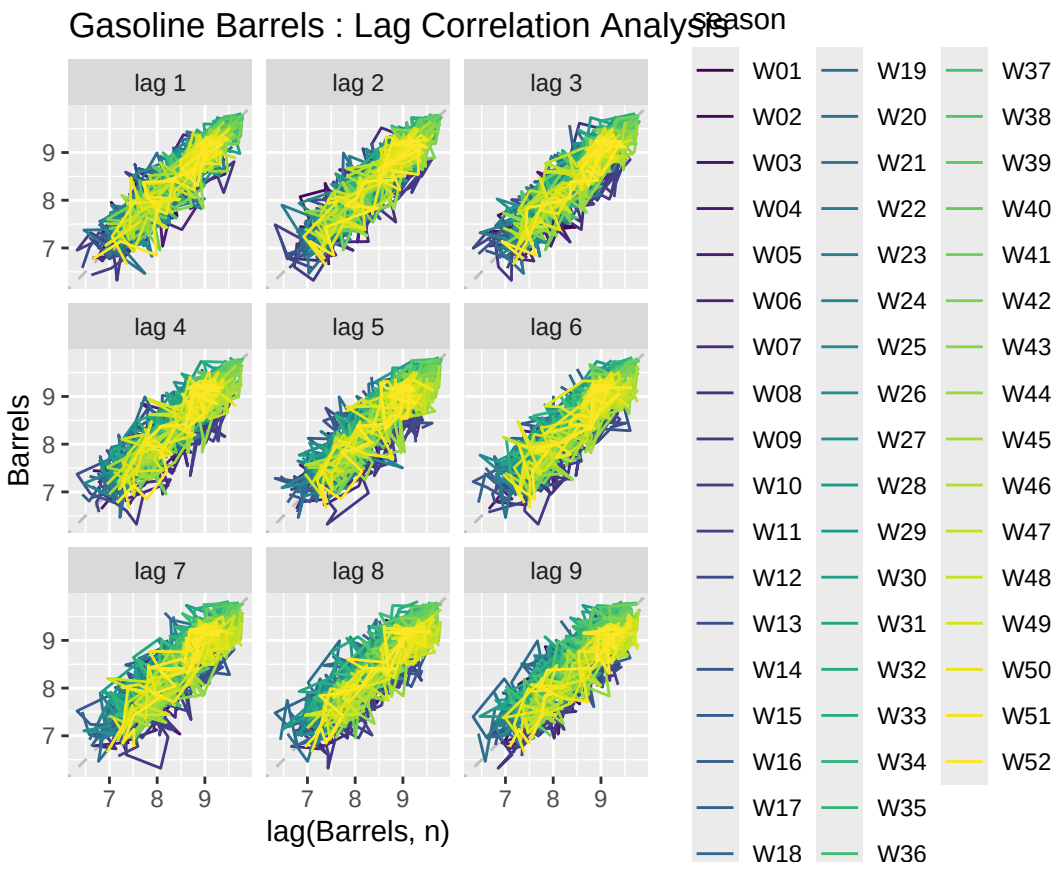

```
gg_season(ga, Barrels, period = "year") +
  ggtitle(make_title("Gasoline Barrels", "Yearly Seasonal Pattern"))
```



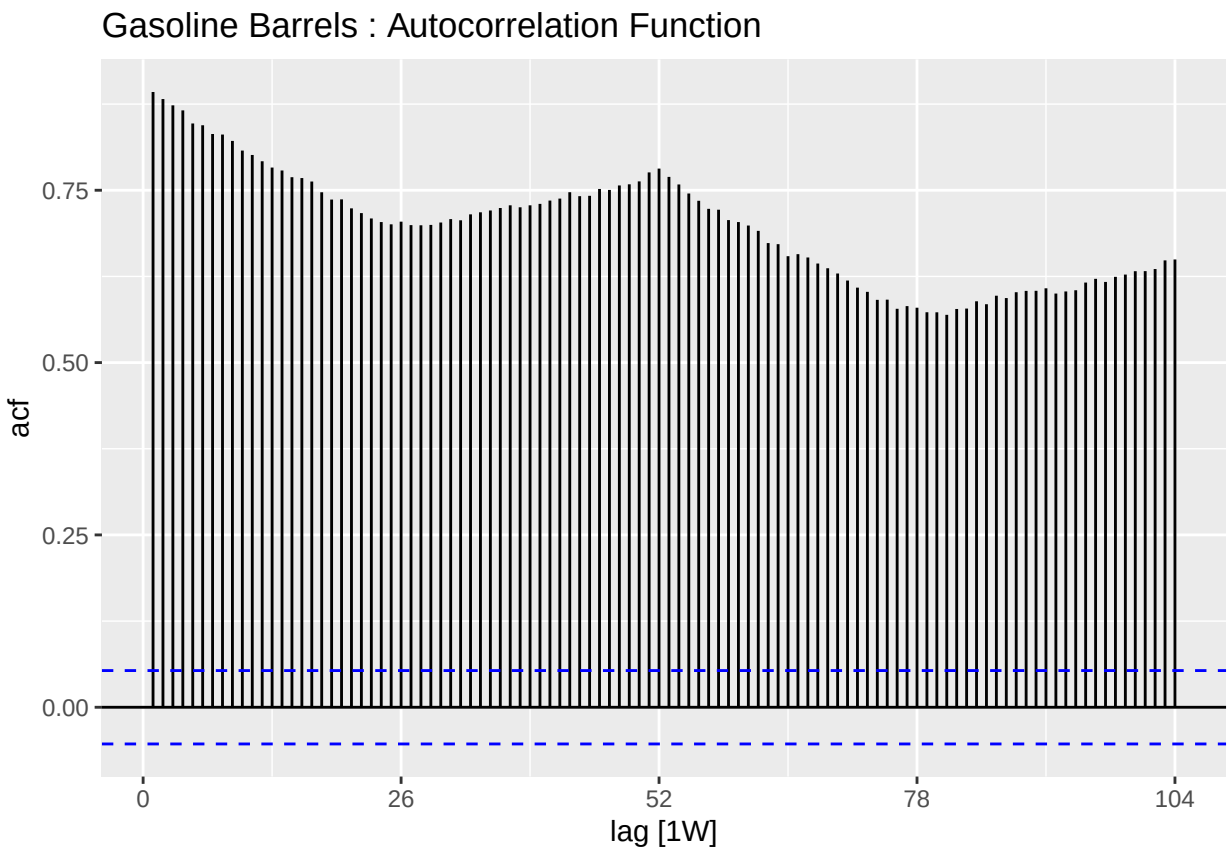

```r
# Create custom weekly visualization
ga %>%
  mutate(Year = year(Week), WeekNum = week(Week)) %>%
  filter(Year >= 2015) %>%  # Recent years
  ggplot(aes(x = WeekNum, y = Barrels, color = as.factor(Year))) +
  geom_line(alpha = 0.7) +
  ggtitle(make_title("Gasoline Barrels", "Week-of-Year Patterns")) +
  labs(color = "Year")
```




```r
gg_lag(ga, Barrels) +
  ggtitle(make_title("Gasoline Barrels", "Lag Correlation Analysis"))
```




```r
ga %>%
  ACF(Barrels, lag_max = 104) %>%  # 2 years of weekly lags
  autoplot() +
  ggtitle(make_title("Gasoline Barrels", "Autocorrelation Function"))
```




**5.1. Trend: COMPLEX Multi-phase, Peaked then declined**

**5.2. Seasonality: MULTIPLE Strong Patterns Yearly: Summer highs** (vacations), winter lows

**5.3. Cyclicity: Economic Demand Cycles 2008 crisis: Massive demand** destruction Recovery phase: 2009-2014 Oil price impacts: Visible in volatility

**5.4. Unusual:** 2020: Pandemic lock down unprecedented drop

**What we learn:**

1. Economic series follow business cycles + seasonality
2. Natural systems (hare) show perfect biological cycles
3. Policy affects healthcare costs in discrete jumps
4. Energy demand has complex multi-frequency patterns